Quantum Neural Networks

Theory, Architectures, and Applications

1 Introduction

1.1 Deep Learning and the Cost of Scale

Over the past decade and a half, deep learning has evolved from a specialized research topic into a general-purpose technology [1]. Convolutional and residual networks transformed image recognition [2, 3], Transformers [4] underpin large language models that translate, summarize, and write code [5], deep networks predict protein structures with near-experimental accuracy [6], and graph neural networks [7] learn from relational data such as molecules and financial transaction graphs. These architectures differ in their inductive biases, yet they share a common foundation: each is a parameterized function approximator whose parameters are learned from data by gradient-based optimization [8, 9].

Much of this progress has been achieved through scale. Empirical scaling laws show that model performance improves predictably, as a power law, with model size, dataset size, and training compute [10], and the compute used to train notable models has doubled roughly every six months since the beginning of the deep learning era [11]. This growth has tangible physical costs [12]: according to the International Energy Agency, global data-center electricity consumption grew by 17% in 2025 and is projected to roughly double by 2030, while the capital expenditure of five large technology companies exceeded $400 billion in 2025, driven largely by data-center investment [13]. These trends raise a natural question: beyond building ever-larger machines that follow classical rules, are there fundamentally different models of computation that could complement them?

1.2 A Different Kind of Computer

In 1982, Feynman observed that simulating quantum systems on classical computers appears to require resources that grow exponentially with system size, and he proposed instead building computers whose components are themselves quantum mechanical [14]. Shor's algorithm factors integers in polynomial time, exponentially faster than the best known classical algorithms [15], and Grover's algorithm offers a quadratic speedup for unstructured search [16]. These algorithms draw on resources that have no classical counterpart: superposition, entanglement, and interference [17]. A register of n qubits is described by 2^n complex amplitudes, so the state space of even a modest quantum computer is far larger than the memory of any classical machine.

This exponential state space, however, does not translate directly into exponential computational power: a measurement of n qubits returns only n classical bits, and the amplitudes can be estimated only statistically through repeated measurements. This tension between a vast internal state and a narrow measurement channel recurs throughout this paper.

On the hardware side, present-day devices belong to what Preskill termed the *noisy intermediate-scale quantum* (NISQ) era: processors with tens to a few hundred qubits that operate without full error correction [18]. In 2019, a 53-qubit superconducting processor performed a random-circuit sampling task that was argued to be intractable for the classical supercomputers of the time [19]. More recently, surface-code experiments demonstrated logical error rates that *decrease* as the code distance is increased from three to five to seven, an essential prerequisite for scalable fault-tolerant quantum computation [20]. Until large-scale

fault-tolerant machines arrive, *variational quantum algorithms* offer a way to use noisy devices of moderate size [21]: a parameterized quantum circuit is executed on quantum hardware, and its parameters are optimized by a classical computer in a hybrid feedback loop.

1.3 Quantum Neural Networks

Quantum machine learning lies at the intersection of these two developments [22, 23], and its most widely studied model is the *quantum neural network* (QNN), also known as a variational or parameterized quantum circuit [24, 25, 26]. The structure of a QNN deliberately mirrors that of a classical neural network. Classical input data $\mathbf{x}$ are encoded into a quantum state, typically as rotation angles; layers of trainable rotations and entangling gates transform this state; and the expectation value of an observable O is measured to produce a classical output:

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \langle 0|^{\otimes n}\, U^{\dagger}(\mathbf{x},\boldsymbol{\theta})\, O\, U(\mathbf{x},\boldsymbol{\theta})\, |0\rangle^{\otimes n}\,. \tag{1.1}$$

The model is trained in the same way as its classical counterpart: a loss function is defined, its gradients with respect to the circuit parameters $\boldsymbol{\theta}$ are estimated on the quantum device [25], and the parameters are updated by a classical optimizer.

QNNs can use the exponentially large Hilbert space as a feature space [27, 28]; a single qubit that repeatedly re-encodes its input can act as a universal classifier [29]; and certain QNNs have been shown to possess a larger effective dimension than comparable classical networks [30]. At the same time, the field faces serious open questions. Randomly initialized circuits can exhibit *barren plateaus*, in which gradients vanish exponentially with the number of qubits and training stalls [31]. Classical learners that have access to data can often match quantum models [32], many QNNs can be closely approximated by classical random-Fourier-feature models [33], and the very structure that allows a circuit to avoid barren plateaus may also allow it to be simulated classically [34]. Whether QNNs offer a practical advantage on classical data therefore remains open, and a more productive starting point is a more fundamental question: *what kind of functions does a QNN compute, and how is this function class shaped by the design of the circuit?*

1.4 A Fourier Perspective

This question is the central theme of this paper, and Fourier analysis provides a remarkably clear answer. Schuld, Sweke, and Meyer [35] showed that, when a scalar input x enters the circuit through gates generated by a Hamiltonian, the output of a QNN is a finite Fourier series:

$$f_{\boldsymbol{\theta}}(x) = \sum_{\omega\in\Omega} c_{\omega}(\boldsymbol{\theta})\, e^{i\omega x}. \tag{1.2}$$

This representation reveals a clean division of labor. The set of frequencies Ω is determined entirely by the eigenvalues of the encoding Hamiltonians and by how many times the input is encoded; the trainable parameters $\boldsymbol{\theta}$ only shape the coefficients c_{ω}. The consequences are striking. A single qubit that encodes x once through the rotation $R_X(x)$ can represent only a single sinusoid, $c_0 + 2|c_1|\cos(x+\varphi)$, no matter how many trainable layers surround the encoding gate. The expressivity of a QNN is therefore governed less by the depth of its

trainable layers than by *how the data are encoded*, which makes the encoding strategy (the number of encoding qubits, the number of re-uploading layers, and the scaling of the input) the principal design lever.

This perspective also invites a direct comparison with classical networks. Multilayer perceptrons exhibit a *spectral bias*: they learn low-frequency components first and struggle to fit high-frequency detail [36]. Classical remedies, such as Fourier feature mappings [37] and periodic activation functions as in SIREN [38], effectively allow the network to learn or select the frequencies it uses. A QNN inverts this arrangement: its frequencies are fixed in advance by the architecture, chosen by the designer rather than discovered during training, and learning adjusts only their amplitudes and phases. Is such a predetermined spectrum a fundamental limitation, since the model can never represent a frequency outside Ω, or is it an advantage, since it provides an explicit, interpretable, and controllable spectral inductive bias? This question runs throughout the paper.

1.5 Scope and Organization

This white paper is intended as a self-contained, pedagogical bridge between classical deep learning and quantum computing for readers familiar with one field and new to the other. Rather than surveying the literature, we construct QNNs from first principles and work through small examples explicitly, so that every claim can be verified step by step. In particular, this paper:

- frames QNNs and multilayer perceptrons as two families of parameterized function approximators, using the universal approximation theorem as a common reference point;
- dissects a standard QNN layer by layer (encoding, trainable rotations, entanglement, and measurement), showing explicitly, for example, how CNOT gates propagate information between qubits;
- presents a complete, step-by-step derivation of the finite Fourier series representation of QNNs; and
- analyzes how parallel encoding, data re-uploading, and input scaling shape the accessible frequency spectrum, together with the resource trade-offs among them.

The remainder of the paper is organized as follows. Section 2 reviews multilayer perceptrons and the universal approximation theorem. Section 3 introduces the essentials of quantum computing and dissects the architecture of a QNN. Section 4 develops the Fourier analysis of QNNs and examines how the encoding strategy determines their expressivity. Section 5 discusses applications.

2 Review of Classical Neural Networks

Before exploring Quantum Neural Networks (QNNs), it is essential to understand their classical counterparts. At their core, neural networks are parameterized mathematical models

designed to approximate complex functions from data. Among the various architectures, the Multilayer Perceptron (MLP) is one of the most fundamental and widely studied. Despite its relatively simple structure, the MLP provides the theoretical foundation for understanding the expressive power of neural networks.

2.1 Architecture of Multilayer Perceptrons

An MLP is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer, as illustrated in Fig. 1 [9, Chap. 6]. Each layer consists of multiple neurons, which perform affine transformations followed by nonlinear activation functions.

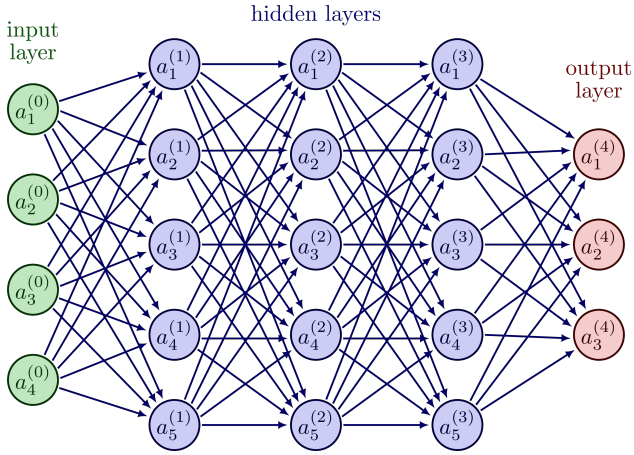


Figure 1: Architecture of a Multilayer Perceptron (MLP), consisting of an input layer, hidden layers, and an output layer.

Mathematically, the output of the l-th hidden layer can be expressed as

$$\mathbf{h}^{(l)} = \sigma\left(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{2.1}$$

where $\mathbf{h}^{(0)} = \mathbf{x}$ denotes the input vector, $W^{(l)}$ and $\mathbf{b}^{(l)}$ are the trainable weight matrix and bias vector, respectively, and σ is a nonlinear activation function. Common choices include Sigmoid, Tanh, and Rectified Linear Unit (ReLU).

By composing multiple layers, an MLP constructs a parameterized mapping from the input space to the output space. Let $A^{(l)}(\mathbf{h}) = W^{(l)}\mathbf{h} + \mathbf{b}^{(l)}$ denote the affine transformation at layer l. An MLP with L affine layers and a linear output layer can then be represented as

$$f_\theta(\mathbf{x}) = A^{(L)} \circ \sigma \circ A^{(L-1)} \circ \cdots \circ \sigma \circ A^{(1)}(\mathbf{x}) \tag{2.2}$$

where θ represents all trainable parameters and $\circ$ denotes function composition.

An important observation is that the nonlinear activation function is essential. Without nonlinearities, any composition of affine transformations remains an affine transformation, regardless of the number of layers. Therefore, nonlinear activations are what enable MLPs to represent functions beyond simple linear mappings.

2.2 MLPs as Function Approximators

The fundamental objective of an MLP is to approximate an unknown target function $f^*(\mathbf{x})$ using a parameterized function $f_\theta(\mathbf{x})$.

To understand how this works, consider a single-hidden-layer MLP with a scalar output:

$$f_\theta(\mathbf{x}) = \sum_{j=1}^{m} a_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j) + c \tag{2.3}$$

where m is the number of hidden neurons, $\mathbf{w}_j$ and b_j determine the transformation applied by the j-th neuron, and a_j controls its contribution to the final output.

This expression reveals a fundamental principle: an MLP approximates a complex function by combining multiple simpler nonlinear functions. Each hidden neuron generates a parameterized basis-like function, and the output layer constructs a weighted combination of these functions.

For example, consider the ReLU activation:

$$\sigma(x) = \max(0, x) \tag{2.4}$$

Each ReLU neuron introduces a piecewise-linear component. By combining sufficiently many such components, an MLP can construct increasingly complex piecewise-linear functions that approximate nonlinear target functions, as illustrated in Fig. 2.

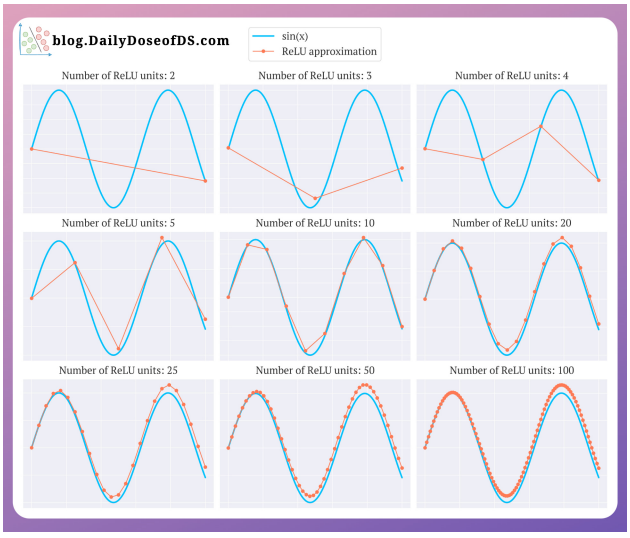


Figure 2: Illustration of function approximation using ReLU-based MLPs. Increasing the number of hidden neurons provides greater flexibility to approximate a nonlinear target function.

Given a training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the network learns its parameters by minimizing a loss function. For example, in regression tasks, the Mean Squared Error (MSE) objective is

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(\mathbf{x}_i) - y_i\right)^2 \tag{2.5}$$

In practice, the parameters are typically optimized through gradient-based methods using backpropagation [8].

2.3 Universal Approximation Theorem

A natural question arises: how complex a function can an MLP approximate?

One of the most important theoretical results in neural network research is the Universal Approximation Theorem (UAT), established through a series of foundational works [39, 40].

In particular, consider a continuous target function $f^* : K \to \mathbb{R}$, where $K \subset \mathbb{R}^d$ is compact. For any approximation tolerance $\epsilon > 0$, there exists a single-hidden-layer ReLU network with sufficiently many hidden neurons such that

$$\boxed{\sup_{\mathbf{x}\in K} |f^*(\mathbf{x}) - f_\theta(\mathbf{x})| < \epsilon} \tag{2.6}$$

In other words, even an MLP with only one hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy, provided that sufficient hidden neurons are available.

This theorem establishes the remarkable expressive potential of neural networks. However, several important distinctions must be made.

First, the theorem guarantees the *existence* of suitable parameters, but does not guarantee that a training algorithm can efficiently discover them. Second, it does not specify how many neurons or parameters are required to achieve a desired approximation accuracy. Finally, approximation capability does not necessarily imply good generalization to unseen data.

Although a single hidden layer is theoretically sufficient for universal approximation, deeper architectures may represent certain functions more efficiently by exploiting their compositional structure [9, 41].

2.4 Beyond MLPs: Toward Quantum Neural Networks

Modern deep learning has extended the basic principles of MLPs into numerous specialized architectures. Convolutional Neural Networks (CNNs) exploit spatial structure, recurrent architectures such as RNNs, LSTMs, and GRUs model sequential dependencies, Transformers leverage attention mechanisms, and Graph Neural Networks (GNNs) incorporate graph structure [9, 4, 7]. Despite their architectural differences, these models share the fundamental objective of learning parameterized functions from data.

This naturally leads to an intriguing question: can quantum computers also serve as function approximators?

While classical MLPs construct functions through compositions of affine transformations and nonlinear activations, variational QNNs construct functions through quantum data encoding, parameterized unitary transformations, and measurements. A typical QNN output can be expressed as [25, 26]

$$f_\theta(\mathbf{x}) = \langle 0|^{\otimes n} U^\dagger(\mathbf{x}, \theta) \mathcal{O} U(\mathbf{x}, \theta) |0\rangle^{\otimes n} \tag{2.7}$$

where $U(\mathbf{x}, \theta)$ represents the parameterized quantum circuit and $\mathcal{O}$ denotes a measurement observable.

Thus, both MLPs and QNNs can be viewed as parameterized function approximators, yet they rely on fundamentally different computational mechanisms. This raises several important questions: How do QNNs represent functions? What determines their expressive power? And how does their function approximation capability compare with that of classical MLPs?

These questions motivate the following investigation into the mathematical representation and expressive properties of Quantum Neural Networks.

3 Quantum Neural Networks

3.1 Basics of Quantum Computing

Quantum computing is built upon the principles of quantum mechanics, which provide a fundamentally different computational framework from classical computing. Unlike classical computers, which process information using binary bits, quantum computers manipulate quantum bits, or qubits, through unitary transformations and quantum measurements. This section introduces the essential concepts of quantum computing, including quantum states, quantum gates, tensor products, entanglement, and measurement. These concepts form the mathematical foundation of Quantum Neural Networks (QNNs). The following discussion is based on the standard quantum computing formalism presented in [17, Chaps. 2 and 4].

3.1.1 Qubits and Quantum States

In classical computing, a bit can take one of two possible values, either 0 or 1. In contrast, a qubit can exist in a superposition of two computational basis states, denoted by $|0\rangle$ and $|1\rangle$. The general state of a single qubit is expressed as

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad |\alpha|^2 + |\beta|^2 = 1 \tag{3.1}$$

where $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes. In vector notation,

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad |\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \tag{3.2}$$

According to the Born rule, the probabilities of measuring $|0\rangle$ and $|1\rangle$ are $|\alpha|^2$ and $|\beta|^2$, respectively. Importantly, the probability amplitudes also contain phase information, which plays a crucial role in quantum interference.

Ignoring an unobservable global phase, any pure single-qubit state can be represented geometrically on the Bloch sphere, as illustrated in Fig. 3:

$$|\psi\rangle = \cos\left(\frac{\vartheta}{2}\right)|0\rangle + e^{i\varphi}\sin\left(\frac{\vartheta}{2}\right)|1\rangle \tag{3.3}$$

where ϑ and φ represent the polar and azimuthal angles, respectively.

For a system containing n qubits, the quantum state belongs to a 2^n-dimensional Hilbert space and can be expressed as

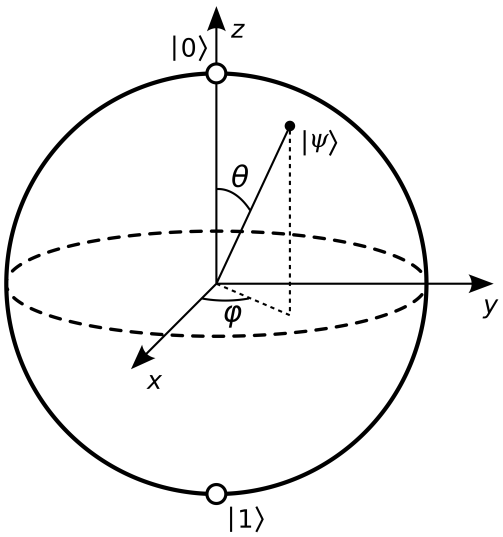


Figure 3: Bloch sphere representation of a single-qubit pure state. The angles ϑ and φ characterize its position on the sphere, up to a global phase.

$$|\psi\rangle = \sum_{j=0}^{2^n-1} a_j \, |j\rangle \,, \qquad \sum_{j=0}^{2^n-1} |a_j|^2 = 1 \tag{3.4}$$

where $|j\rangle$ denotes the computational basis state corresponding to the binary representation of j.

Throughout this paper, we adopt the *little-endian convention*, in which the computational basis is ordered as $|q_{n-1} \cdots q_1 q_0\rangle$, with q_0 being the least significant bit.

Although the dimension of the quantum state space grows exponentially with the number of qubits, this does not automatically imply an exponential computational advantage, since extracting information from a quantum state is constrained by quantum measurement.

3.1.2 Quantum Gates and Unitary Operations

Quantum states are manipulated through quantum gates, which are mathematically represented by unitary matrices. A quantum gate U transforms an initial state $|\psi\rangle$ into

$$|\psi'\rangle = U \, |\psi\rangle \,, \qquad U^\dagger U = I \tag{3.5}$$

where $U^\dagger$ is the conjugate transpose of U. The unitary condition ensures that the normalization of the quantum state is preserved throughout the evolution of an ideal quantum circuit.

Several fundamental single-qubit gates are widely used in quantum computing. For example, the Pauli-X gate acts similarly to a classical NOT gate, while the Hadamard gate transforms computational basis states into equal superpositions:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{3.6}$$

In particular,

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \tag{3.7}$$

Another important class of gates is the rotation gates, which rotate quantum states around different axes of the Bloch sphere. They can be expressed as

$$R_\nu(\theta) = e^{-i\theta\sigma_\nu/2}, \qquad \nu \in \{x, y, z\} \tag{3.8}$$

where σ_ν denotes the corresponding Pauli operator. For example, the rotation around the X-axis is

$$R_x(\theta) = \begin{bmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix} \tag{3.9}$$

Rotation gates are particularly important in QNNs because their rotation angles can encode classical input features or represent trainable model parameters.

When multiple qubits are involved, the combined state space is constructed using the tensor product. If independent single-qubit gates $U_0, U_1, \ldots, U_{n-1}$ act on their respective qubits, the overall operation is represented, under our little-endian convention, as

$$U = U_{n-1} \otimes \cdots \otimes U_1 \otimes U_0 \tag{3.10}$$

The tensor product provides a systematic way to construct multi-qubit operations from individual gates. However, not all multi-qubit operations can be decomposed into tensor products of single-qubit gates. Such operations are essential for generating quantum entanglement.

3.1.3 Multi-Qubit Gates and Entanglement

One of the most fundamental multi-qubit gates is the Controlled-NOT (CNOT) gate, which operates on two qubits: a control qubit and a target qubit. The target is flipped if and only if the control is in state $|1\rangle$. Its action on computational basis states is defined as

$$|c, t\rangle \xrightarrow{\text{CNOT}} |c, t \oplus c\rangle \tag{3.11}$$

where $\oplus$ denotes modulo-2 addition.

For a two-qubit system following the little-endian convention $|q_1 q_0\rangle$, if q_1 is the control and q_0 is the target, the corresponding matrix is

$$\text{CNOT}_{q_1 \to q_0} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{3.12}$$

For example, $\text{CNOT}_{q_1 \to q_0}|10\rangle = |11\rangle$.

Conversely, if q_0 is the control and q_1 is the target, the matrix becomes

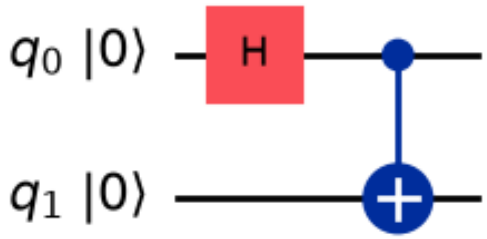


Figure 4: Generation of a Bell state using a Hadamard gate followed by a CNOT gate. The resulting two-qubit state is maximally entangled.

$$\text{CNOT}_{q_0 \to q_1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \tag{3.13}$$

Notice that reversing the control and target qubits changes the matrix representation. For instance,

$$\text{CNOT}_{q_0 \to q_1} |01\rangle = |11\rangle \tag{3.14}$$

Beyond conditional bit flipping, the CNOT gate plays a crucial role in creating quantum entanglement. Consider two qubits initially prepared in $|00\rangle$. As illustrated in Fig. 4, we first apply a Hadamard gate to q_0:

$$\begin{aligned} |\psi\rangle &= (I \otimes H) |00\rangle \\ &= \frac{1}{\sqrt{2}} (|00\rangle + |01\rangle) \end{aligned} \tag{3.15}$$

Next, applying a CNOT gate with q_0 as the control and q_1 as the target produces

$$\begin{aligned} |\psi_{\text{Bell}}\rangle &= \text{CNOT}_{q_0 \to q_1} |\psi\rangle \\ &= \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) \end{aligned} \tag{3.16}$$

The resulting Bell state is entangled, meaning that it cannot be expressed as the tensor product of two individual single-qubit states. Measuring both qubits in the computational basis produces either $|00\rangle$ or $|11\rangle$, each with probability $1/2$.

More generally, for an arbitrary two-qubit pure state

$$|\psi\rangle = a |00\rangle + b |01\rangle + c |10\rangle + d |11\rangle \tag{3.17}$$

entanglement can be determined by examining the coefficient matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \tag{3.18}$$

The state is separable if and only if A has rank one, or equivalently, its determinant vanishes:

$$\boxed{\begin{cases} ad - bc = 0, & \text{Separable} \\ ad - bc \neq 0, & \text{Entangled} \end{cases}} \tag{3.19}$$

This criterion applies specifically to two-qubit pure states. For the Bell state in Eq. 3.16, $ad - bc = 1/2 \neq 0$, confirming that the state is entangled.

Entangling gates such as CNOT are important building blocks of QNNs, as they enable quantum circuits to establish interactions among qubits that cannot be generated by independent single-qubit operations alone.

3.1.4 Quantum Measurement and Connection to QNNs

Unlike classical computation, where the values of computational variables can generally be accessed directly, extracting information from a quantum system requires measurement.

For a single qubit $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, a computational-basis measurement yields either 0 or 1, with probabilities

$$P(0) = |\alpha|^2, \qquad P(1) = |\beta|^2 \tag{3.20}$$

A single measurement does not reveal the underlying complex probability amplitudes. Instead, repeated preparation and measurement of the same quantum state allow us to estimate measurement probabilities and expectation values statistically.

In particular, the expectation value of a Hermitian observable $\mathcal{O}$ for a quantum state $|\psi\rangle$ is

$$\langle \mathcal{O} \rangle = \langle \psi | \mathcal{O} | \psi \rangle \tag{3.21}$$

This provides a natural mechanism for converting quantum information into classical numerical outputs.

The concepts introduced in this section form the foundation of variational QNNs. In a typical QNN, classical input data $\mathbf{x}$ are encoded into quantum states, trainable quantum gates transform these states, entangling operations introduce interactions among qubits, and quantum measurements produce classical outputs.

Mathematically, such a model can be expressed as

$$|\psi(\mathbf{x}, \theta)\rangle = U(\mathbf{x}, \theta) |0\rangle^{\otimes n} \tag{3.22}$$

with the corresponding output function

$$f_\theta(\mathbf{x}) = \langle 0|^{\otimes n} U^\dagger(\mathbf{x}, \theta) \mathcal{O} U(\mathbf{x}, \theta) |0\rangle^{\otimes n} \tag{3.23}$$

where θ denotes the trainable circuit parameters.

Thus, quantum states, unitary operations, entanglement, and measurement collectively provide the building blocks for constructing parameterized quantum models. The following section examines how these principles are combined to form Quantum Neural Networks and how quantum circuits can be used for function approximation.

3.2 Dissection of QNN

Similar to classical neural networks, quantum neural networks (QNN) can also serve as function approximators, with universality requiring suitable encoding and circuit families [25, 29, 35]. Figure 5 shows a **3-qubit** circuit architecture of a standard quantum neural networks. Simply speaking, it is composed of layers of rotation gate (X,Y,Z) and CNOT gate. In the end, the each wire has a measurement gate (usually Z-basis). Let's look at each gate

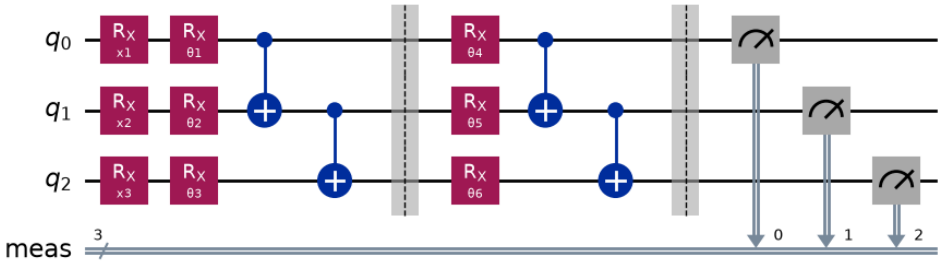


Figure 5: A standard quantum neural network

one by one. Firstly, there are three types of rotation gate R_x, R_y, R_z. Any single-qubit gate is a 2×2 unitary matrix and these three matrices, variants of Pauli-matrices, rotates the vector in the bloch sphere about x, y, z axis by θ degrees. Their mathmatical definition is as follows:

$$R_x(\theta) \equiv e^{-i\theta X/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})X = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}. \tag{3.24}$$

$$R_y(\theta) \equiv e^{-i\theta Y/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})Y = \begin{bmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}. \tag{3.25}$$

$$R_z(\theta) \equiv e^{-i\theta Z/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})Z = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}. \tag{3.26}$$

Controlled-NOT (CNOT) gate has two input qubits, known as control qubit and target qubit, respectively. In terms of the computational basis, the action of CNOT is given by $|c\rangle |t\rangle \rightarrow |c\rangle |t \oplus c\rangle$. That is if the control qubit is set to $|1\rangle$ then the target qubit is flipped, otherwise the target qubit is left alone. Thus, in the computational basis $|contro, target\rangle$ the matrix representation of CNOT is:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

For example, if a two-qubit system is in state $|11\rangle$, which in basis vector is $\begin{bmatrix}0\\0\\0\\1\end{bmatrix}$. Here we follow little-endian (LSB)convention ($|q_1 q_0\rangle$) and let $|q_0\rangle$ be the control qubit and $|q_1\rangle$ be the target qubit. When a CNOT gate is applied to it:

$$\begin{bmatrix}1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0\end{bmatrix} \times \begin{bmatrix}0\\0\\0\\1\end{bmatrix} = \begin{bmatrix}0\\1\\0\\0\end{bmatrix} = |01\rangle$$

$$CNOT\,|11\rangle \rightarrow |01\rangle$$

These three rotation gates and CNOT gate are also known universal gates in quantum computer, similar to NOR and NAND in classical computer. Up to an overall global phase, any unitary matrix can be decomposed into a sequence of single-qubit rotation gates and CNOT gates [42]. Similar to classical neural network, quantum neural network also has input, hidden and output layers, which corresponds to encoding, rotation and entanglment in Hilbert Space, and measurement. Let's look at them one by one.

3.3 Input Layer: Encoding

For most of machine learning tasks, the data is classical. Therefore, we need a way to encode the classical data into quantum states. Three main encoding approaches are [26]:

1. **Amplitude Encoding**
2. **Basis Encoding**
3. **Angular Encoding**

Of course, there are other more advanced encoding such as Pauli Encoding and IQP Encoding but it's not in the scope of this article. Here, we first talk about the most common encoding method - **Angular Encoding** because it's the easiest to prepare from engineering perspective. Just as the name suggests, the classical data x_i is mapped into a rotation angle using rotation gates. Figure 6 shows how a 3-dimension input vector is embedded into a quantum circuit. Each qubit, represented by a single line here, starts with 0 basis state $|0\rangle$. Then qubit is followed by a rotation gate (can be R_x, R_y, R_z). These rotation gates will then be parametrized by the input values x_1, x_2, x_3. For example, for the first qubit (little-endian), the starting state is $|0\rangle = \begin{bmatrix}1\\0\end{bmatrix}$. Then the rotation gate, let's say R_x, is parametrized by x_1:

$$R_x(x_1) = \begin{bmatrix}\cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2})\\ -i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2})\end{bmatrix}.$$

Then the qubit is operated as:

$$R_x(x_1)\,|0\rangle = \begin{bmatrix}\cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2})\\ -i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2})\end{bmatrix} \times \begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}\cos(\frac{x_1}{2})\\ -i\sin(\frac{x_1}{2})\end{bmatrix}$$

This is equivalent to:

$$|0\rangle \rightarrow \cos(\frac{x_1}{2})\,|0\rangle + (-i\sin(\frac{x_1}{2}))\,|1\rangle$$

If you use R_y, then you will have:

$$R_y(x_1)\,|0\rangle = \begin{bmatrix} \cos(\frac{x_1}{2}) & -\sin(\frac{x_1}{2}) \\ \sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2}) \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos(\frac{x_1}{2}) \\ \sin(\frac{x_1}{2}) \end{bmatrix}$$

Similarly, for the rest of input values x_2, x_3, x_4, the quantum states of the qubits become (assume we use R_x):

$$q_1(x_2) = \begin{bmatrix} \cos(\frac{x_2}{2}) \\ -i\sin(\frac{x_2}{2}) \end{bmatrix}, \quad q_2(x_3) = \begin{bmatrix} \cos(\frac{x_3}{2}) \\ -i\sin(\frac{x_3}{2}) \end{bmatrix}$$

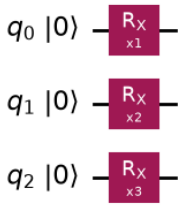


Figure 6: Input layer of a QNN using angluar encoding.

If we look at the 3-qubit system holistically, which is also known as product state, the state transitions from $|000\rangle$ to $|q_2q_1q_0\rangle$. Mathmatically as follows:

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \xrightarrow{\text{angle encoding}} \begin{bmatrix} \cos(\frac{x_3}{2}) \\ -i\sin(\frac{x_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{x_2}{2}) \\ -i\sin(\frac{x_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{x_1}{2}) \\ -i\sin(\frac{x_1}{2}) \end{bmatrix}$$

This is equivalent to:

$$|q_2q_1q_0\rangle = |q_2\rangle \otimes |q_1\rangle \otimes |q_0\rangle$$

The statevector of entire circuit can be rewritten into tensor product of statevector of each single qubit becuase entanglement has not yet formed.

The tensor product of three 2×1 vectors is a 8×1 vector, which is expanded as follows:

$$\psi(x_3, x_2, x_1) = \begin{bmatrix} \cos(\frac{x_3}{2})\cos(\frac{x_2}{2})\cos(\frac{x_1}{2}) \\ -i\cos(\frac{x_3}{2})\cos(\frac{x_2}{2})\sin(\frac{x_1}{2}) \\ -i\cos(\frac{x_3}{2})\sin(\frac{x_2}{2})\cos(\frac{x_1}{2}) \\ -\cos(\frac{x_3}{2})\sin(\frac{x_2}{2})\sin(\frac{x_1}{2}) \\ -i\sin(\frac{x_3}{2})\cos(\frac{x_2}{2})\cos(\frac{x_1}{2}) \\ -\sin(\frac{x_3}{2})\cos(\frac{x_2}{2})\sin(\frac{x_1}{2}) \\ -\sin(\frac{x_3}{2})\sin(\frac{x_2}{2})\cos(\frac{x_1}{2}) \\ i\sin(\frac{x_3}{2})\sin(\frac{x_2}{2})\sin(\frac{x_1}{2}) \end{bmatrix}$$

If we look each qubit individually, the encoding is a 2×2 matrix. However, if we look the encoding holistically on the entire 3 qubit system, it's a $2^3 \times 2^3 = 8 \times 8$ matrix, which is also a tensor product of all three rotation matrices:

$$\begin{bmatrix} \cos(\frac{x_3}{2}) & -i\sin(\frac{x_3}{2}) \\ -i\sin(\frac{x_3}{2}) & \cos(\frac{x_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{x_2}{2}) & -i\sin(\frac{x_2}{2}) \\ -i\sin(\frac{x_2}{2}) & \cos(\frac{x_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2}) \\ -i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2}) \end{bmatrix}$$

This makes sense becuase with 3 qubits, the quantum state is in $\mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^8$ and the operation shall also be a 8×8 matrix as follows. To make it readable, let's define:

$$c_j = \cos\left(\frac{x_j}{2}\right), \qquad s_j = \sin\left(\frac{x_j}{2}\right), \qquad j = 1, 2, 3, 4.$$

$$U = R_x(x_3) \otimes R_x(x_2) \otimes R_x(x_1)$$

The matrix is as follows:

$$\begin{bmatrix} c_3c_2c_1 & -ic_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 \\ -ic_3c_2s_1 & c_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 \\ -ic_3s_2c_1 & -c_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 \\ -c_3s_2s_1 & -ic_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 \\ -is_3c_2c_1 & -s_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 \\ -s_3c_2s_1 & -is_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 \\ -s_3s_2c_1 & is_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 \\ is_3s_2s_1 & -s_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 \end{bmatrix} \tag{3.27}$$

Please be aware that since we use little-endian convention, the tensor product of operation matrix follows the same order as the qubit. If our initial state is $|0\rangle$, then the resulting state is the first column of this matrix. In fact, the matrix is

$$U_{\mathbf{b},\mathbf{b}'} = (-i)^{d_H(\mathbf{b},\mathbf{b}')} \prod_{j:b_j=b'_j} c_j \prod_{j:b_j\neq b'_j} s_j$$

where $d_H(\mathbf{b}, \mathbf{b}')$ is Hamming distance between target state and initial state.
Before concluding angle encoding, there is one more thing - **scaling**. As you might have observed, the trignometric functions are all periodic (2π) meaning that

$$|\psi(x)\rangle = |\psi(x + 2\pi)\rangle$$

If the range of classical data is larger than 2π, then two different classical values will be mapped to the same quantum state, causing information loss. Therefore, scaling must be conducted such that the input values, before encoding, must be between 0 and π. This is becuase the periodicity of cosine and sine functions are 2π, so the periodicity in $\cos(\frac{x}{2})$ is π. A simple and most common way to do scaling is as follows:

$$\theta = \pi \frac{x - x_{min}}{x_{max} - x_{min}}$$

Now, the features are mapped into a non-repeatitive and continuous angle space. Finally, there are many variants of angle encoding such as XYZ encoding, data-reuploading, multiscale, and adaptive encoding. What has to be aware of is that encoding can have notrivial impact on the representation capacity of the model [35, 29].

3.4 Hidden Layer - Rotation

Now, after angle encoding, each qubit is in the state

$$|\psi\rangle \cos(\frac{x_i}{2}) |0\rangle + (-i \sin(\frac{x_i}{2})) |1\rangle = \begin{bmatrix} \cos(\frac{x_1}{2}) \\ -i \sin(\frac{x_1}{2}) \end{bmatrix}.$$

Then, this qubit will go through another rotation gate, let's say R_x, which is parametrized by θ, resembling the weight in classical neural networks. The state becomes:

$$R_x(\theta) |\psi\rangle = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i \sin(\frac{\theta}{2}) \\ -i \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \times \begin{bmatrix} \cos(\frac{x_i}{2}) \\ -i \sin(\frac{x_i}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta}{2}) \cos(\frac{x_i}{2}) - \sin(\frac{\theta}{2}) \sin(\frac{x_i}{2}) \\ -i \sin(\frac{\theta}{2}) \cos(\frac{x_i}{2}) - i \cos(\frac{\theta}{2}) \sin(\frac{x_i}{2}) \end{bmatrix}$$

Using following trignometric identity, we can further simplify.

$$\cos(A + B) = \cos(A) \cos(B) - \sin(A) \sin(B)$$
$$\sin(A + B) = \sin(A) \cos(B) + \cos(A) \sin(B)$$

$$\begin{bmatrix} \cos(\frac{\theta}{2}) \cos(\frac{x_i}{2}) - \sin(\frac{\theta}{2}) \sin(\frac{x_i}{2}) \\ -i \sin(\frac{\theta}{2}) \cos(\frac{x_i}{2}) - i \cos(\frac{\theta}{2}) \sin(\frac{x_i}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta + x_i}{2}) \\ -i \sin(\frac{\theta + x_i}{2}) \end{bmatrix}$$

For example, in Figure 7, the first qubit becomes:

$$|q_0\rangle = \begin{bmatrix} \cos(\frac{\theta_1 + x_1}{2}) \\ -i \sin(\frac{\theta_1 + x_1}{2}) \end{bmatrix}.$$

For the rest of qubits, the states becomes:

$$q_1 = \begin{bmatrix} \cos(\frac{\theta_2 + x_2}{2}) \\ -i \sin(\frac{\theta_2 + x_2}{2}) \end{bmatrix}, \quad q_2 = \begin{bmatrix} \cos(\frac{\theta_3 + x_3}{2}) \\ -i \sin(\frac{\theta_3 + x_3}{2}) \end{bmatrix}$$

The entire 3-qubit system then becomes the tensor product of these four vectors:

$$\psi_{q_3q_2q_1} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_2+x_2}{2}) \\ -i\sin(\frac{\theta_2+x_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix}$$

Which can be expaned as:

$$\begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.28}$$

Similarily, the operation matrix on these 3-qubits system is also the tensor product of these three 2×2 matrices:

$$U_{hidden} = \begin{bmatrix} \cos(\frac{\theta_3}{2}) & -i\sin(\frac{\theta_3}{2}) \\ -i\sin(\frac{\theta_3}{2}) & \cos(\frac{\theta_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_2}{2}) & -i\sin(\frac{\theta_2}{2}) \\ -i\sin(\frac{\theta_2}{2}) & \cos(\frac{\theta_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_1}{2}) & -i\sin(\frac{\theta_1}{2}) \\ -i\sin(\frac{\theta_1}{2}) & \cos(\frac{\theta_1}{2}) \end{bmatrix}$$

Similarily, if we define:

$$c_j = \cos(\frac{\theta_j}{2}), \quad s_j = \sin(\frac{\theta_j}{2}), \quad j = 1, 2, 3, 4$$

Then, this 8×8 matrix is the same as Eq. (3.27)

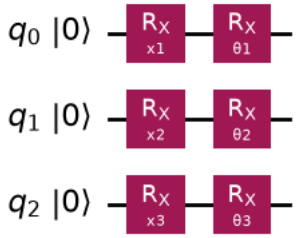


Figure 7: First hidden layer of rotation gates

3.5 Hidden Layer - Entanglement

Local rotations and entangling gates are standard building blocks of parameterized quantum circuits [26]. What's getting more interesting is the CNOT gate in the next layer. If we

only apply rotation gates on each qubit separately, there is no interactions between qubits. CNOT gate allows interactions between qubits so that information can be passed across qubits instead of just staying at only a single qubit. A good analogy is that in classical neural networks, each neuron takes all the neurons in the previous layer as inputs, if we assume fully-connected layers.
Although we have a 3-qubit system, but let's look at the first two qubits q_0, q_1. In Figure 8, after encoding layer and first rotation layer, the state of each qubit is:

$$\begin{aligned} |q_0\rangle &= \cos(\frac{\theta_1 + x_1}{2}) |0\rangle + (-i \sin(\frac{\theta_1 + x_1}{2})) |1\rangle \\ |q_1\rangle &= \cos(\frac{\theta_2 + x_2}{2}) |0\rangle + (-i \sin(\frac{\theta_2 + x_2}{2})) |1\rangle \end{aligned} \tag{3.29}$$

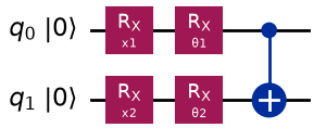


Figure 8: CNOT gate layer on two qubits after rotation gates

The state of entire system is:

$$|q_1 q_0\rangle = \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \cos(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_2+x_2}{2}) \sin(\frac{\theta_1+x_1}{2}) \\ -i \cos(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -\sin(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \end{bmatrix} \tag{3.30}$$

If we apply a CNOT gate on these two qubits, then we have:

$$CNOT\,|q_1 q_0\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \cos(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_2+x_2}{2}) \sin(\frac{\theta_1+x_1}{2}) \\ -i \cos(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -\sin(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \cos(\frac{\theta_2+x_2}{2}) \\ -\sin(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_2+x_2}{2}) \sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.31}$$

As you might observe, the second element and the fourth element of state vector got swapped. Now, these two qubits are entangled! This means that the system cannot be rewriten into a tensor product of two single qubit anymore. You can easily verify this by yourself using Eq. **??**.

$$CNOT\,|q_1 q_0\rangle \neq |q_1\rangle \otimes |q_0\rangle$$

Now, if take the third qubit into consideration and look at the entire system, the operation matrix is:

$$U_{entangle} = I \otimes CNOT_{q_0 \to q_1} \tag{3.32}$$

where I is 2×2 identity matrix. To expand, we have

$$
\begin{aligned}
U_{entangle} &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}
\end{aligned}
\tag{3.33}
$$

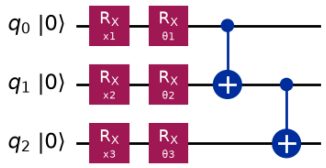


Figure 9: Entanglement layer in QNN

Then, we can apply this matrix 3.33 on the statevector we derived from Eq 3.28:

$$
\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix}
\tag{3.34}
$$

Please notice that the original order $(1, 2, 3, 4, \quad 5, 6, 7, 8)$ becomes $(1, 4, 3, 2, \quad 5, 8, 7, 6)$. For every "subgroup", the second and fourth element are swapped.
Then, as Figure 9 shows, we apply CNOT gate on the second and third qubits q_1, q_2, the

operation matrix is:

$$U_{entangle} = CNOT_{q_1 \to q_2} \otimes I$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Then, we apply this matrix on the statevector:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.35}$$

If you look carefully, What happens is that compare with Eq. 3.34, swapping happens as follows:

$$3 \leftrightarrow 7, \quad 4 \leftrightarrow 8$$

As you might observe, you can apply a CNOT gate on any of the two qubits (e.g., q_0, q_2) and CNOT gates are essentially just permutation matrix that induces a permutation of amplitude in statevector. The magic is that all these four qubits are entangled meaning the statevector is not simply a tensor product of four single-qubit statevector:

$$|q_2 q_1 q_0\rangle \neq |q_2\rangle \otimes |q_1\rangle \otimes |q_0\rangle$$

3.6 Information Propogation through CNOT Gates

The importance of CNOT gate is allowing information be propogatied from control qubit to target qubit, if we measure in Z-basis. Let's look at a 2-qubit system first. Previously, we have derived states of 2-qubit system in Eq. 3.30 that after a layer of encoding and rotation.

The probability of observing $q_1 = 0$ is as follows.

$$\begin{aligned} \text{Let} \quad \alpha = \frac{x_1 + \theta_1}{2}, \quad \beta = \frac{x_2 + \theta_2}{2} \\ P(|q_1\rangle = |0\rangle) &= \cos^2(\alpha)\cos^2(\beta) + \cos^2(\beta)\sin^2(\alpha) \\ &= \cos^2(\beta)(\cos^2(\alpha) + \sin^2(\alpha)) \\ &= \cos^2(\beta) = \cos(\frac{x_2 + \theta_2}{2}) \end{aligned} \tag{3.36}$$

As you can see, the probability of each state in q_1 is a function of $x2$ and θ_2:

$$P(|q_1\rangle = |0\rangle) = f(x_2, \theta_2)$$

However, after a CNOT gate, the statevector becomes Eq. 3.31. The The probability of observing $q_1 = 0$ becomes as follwoing.

$$\begin{aligned} P(|q_1\rangle = |0\rangle) &= \cos^2(\alpha)\cos^2(\alpha) + \sin^2(\alpha)\sin^2(\beta) \\ &= \cos^2(\frac{x_1 + \theta_1}{2})\cos^2(\frac{x_2 + \theta_2}{2}) + \sin^2(\frac{x_1 + \theta_1}{2})\sin^2(\frac{x_2 + \theta_2}{2}) \\ &= f(x_1, x_2, \theta_1, \theta_2) \end{aligned} \tag{3.37}$$

Now, as you can see, the probability amplitude of q_1 becomes a function of $x_1, x_2, \theta_1, \theta_2$! The information from q_0, the first qubit, both input value x_1 and parameters θ_1 are passed down to q_1. This gives the circuits much greater expressivity. However, another thing to notice is that the probability of q_0 does not change before or after CNOT gate.

$$P(|q_0\rangle = |0\rangle) = \cos^2(\frac{\theta_1 + x_1}{2}) \tag{3.38}$$

Intrestingly, the infomration did not pass from target to control. However, this is given we measure in Z-basis. Measurement in other basis will lead to different sitautions.
Again, if we compare with classical neural networks, it's a function of all the input values as Eq. 3.39 shows. This is also knonw as fully connected layers. If the output of a classical Nueral Networks is function of only a single input x_i, it will not be able learn the latent structure of the data, significantly harming the expressivity. The entanglement layer tries to do the similar thing.

$$\begin{aligned} f_{NN} &= f(\sigma(W^T X + b)) \\ &= f(\sigma(\sum_{n=1}^{N} w_n x_n + b)) \\ &= f(x_1, x_2, \cdots, x_N, W) \end{aligned} \tag{3.39}$$

Now, let's look at the 3-qubit system. Before the entanglment layer, the probability of observing the fourth qubit $|q_2\rangle = |0\rangle$ can be derived from Eq. 3.28 as follows:

$$\begin{aligned}
\text{Let} \alpha_j &= \frac{x_j + \theta_j}{2}, \quad j = 1, 2, 3 \\
P(q_2 = 0) &= |\psi_{000}|^2 + |\psi_{001}|^2 + |\psi_{010}|^2 + |\psi_{011}|^2 \\
&= \cos^2(\alpha_3)\cos^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\cos^2(\alpha_2)\sin^2(\alpha_1) \\
&+ \cos^2(\alpha_3)\sin^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\sin^2(\alpha_2)\sin^2(\alpha_1) \\
&= \cos^2(\alpha_3)[\cos^2(\alpha_2)(\cos^2(\alpha_1) + \sin^2(\alpha_1)) + \sin^2(\alpha_2)(\cos^2(\alpha_1) + \sin^2(\alpha_1))] \\
&= \cos^2(\alpha_3) \\
&= \cos^2(\frac{\theta_3 + x_3}{2})
\end{aligned} \tag{3.40}$$

As you can see, without CNOT (entanglement) layer, regardless how many rotation gate you add to the third qubit q_2 (this also applied to other qubits), the probability amplitude for each qubit is just a function of its encoding input x_3 and the parameters of the rotation gate in its own path θ_3. Mathmatically, it's $P(q_2 = 0) = f(\theta_3, x_3)$ The information will never be propagated across the lines. However, after the first entanglement layer, according to Eq. 3.35, the the probability amplitude of $|q_3\rangle = |0\rangle$ becomes as follows:

$$\begin{aligned}
P(q_2 = 0) &= \cos^2(\alpha_3)\cos^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\sin^2(\alpha_2)\sin^2(\alpha_1) \\
&+ \sin^2(\alpha_3)\sin^2(\alpha_2)\cos^2(\alpha_1) + \sin^2(\alpha_3)\cos^2(\alpha_2)\sin^2(\alpha_1) \\
&= \cos^2(\alpha_3)[\cos^2(\alpha_2)\cos^2(\alpha_1) + \sin^2(\alpha_2)\sin^2(\alpha_1)] \\
&+ \sin^2(\alpha_3)[\sin^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_2)\sin^2(\alpha_1)] \\
&= \frac{\cos^2(\alpha_3)}{2}[1 + \cos(2\alpha_2)\cos(2\alpha_1)] + \frac{\cos^2(\alpha_3)}{2}[1 - \cos(2\alpha_2)\cos(2\alpha_1)] \\
&= \frac{1}{2}[1 + \cos(2\alpha_3)\cos(2\alpha_2)\cos(2\alpha_1)] \\
&= \frac{1}{2}[1 + \cos(\theta_3 + x_3)\cos(\theta_2 + x_2)\cos(\theta_1 + x_1)] \\
&= \frac{1}{2}[1 + \prod_{j=1}^{3}\cos(\theta_j + x_j)]
\end{aligned} \tag{3.41}$$

If we look back at, it's clear that

$$\begin{aligned}
P_{\text{before}}(q_2 = 0) &= \cos^2(\frac{\theta_3 + x_3}{2}) \\
P_{\text{after}}(q_2 = 0) &= \frac{1}{2}[1 + \prod_{j=1}^{3}\cos(\theta_j + x_j)]
\end{aligned} \tag{3.42}$$

Now, you can clearly see what magic the entanglment layer has done. If you measure q_3 before entanglment, its probability amplitude is determined by only θ_4, x_4 whereas after entanglment it's determined by all the inputs and parameters. As said before, after entanglment, the statevector cannot be rewritten into tensor product of statevector of each single qubit.

Therefore, all the computation after this point will be holistic, meaning that we need to do $(2^n \times 2^n) \times (2^n \times 1)$ matrix-vector calculation at each layer, which is a big challenge for classical computer. Imagine if you have 30 qubits, the dimension is 1,073,741,924. None of the CPUs, GPUs, or TPUs has enough RAM to do this calculation. However, it can be easily done by a qunatum computer, in fact IBM machines such as Nighthawk or Heron family offer 120 qubits and this number will be increasing in the next couple of years. In terms of large matrix calculation, the quantum advantage is clear.
Alright, let's keep watching how this quantum neural network evolve. As Figure 10 shows, another rotation layer is applied after entanglement layer, as the second hidden layer. The rotation gates are parametrized as $\theta_4, \theta_5, \theta_6$ respectively. The operation matrix of this layer

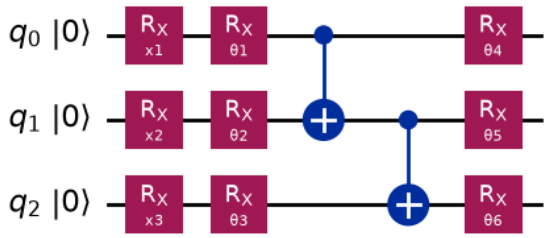


Figure 10: Quantum Neural Network with 2 hidden layers

is then:

$$
\begin{aligned}
U_{hidden} &= R_x(\theta_6) \otimes R_x(\theta_5) \otimes R_x(\theta_4) \\
&= \begin{bmatrix} \cos(\frac{\theta_6}{2}) & -i\sin(\frac{\theta_6}{2}) \\ -i\sin(\frac{\theta_6}{2}) & \cos(\frac{\theta_6}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_5}{2}) & -i\sin(\frac{\theta_5}{2}) \\ -i\sin(\frac{\theta_5}{2}) & \cos(\frac{\theta_5}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_4}{2}) & -i\sin(\frac{\theta_4}{2}) \\ -i\sin(\frac{\theta_4}{2}) & \cos(\frac{\theta_4}{2}) \end{bmatrix} \\
&= \begin{bmatrix}
c_6c_5c_4 & -ic_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 \\
-ic_6c_5s_4 & c_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 \\
-ic_6s_5c_4 & -c_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 \\
-c_6s_5s_4 & -ic_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 \\
-is_6c_5c_4 & -s_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 \\
-s_6c_5s_4 & -is_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 \\
-s_6s_5c_4 & is_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 \\
is_6s_5s_4 & -s_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4
\end{bmatrix}
\end{aligned}
\tag{3.43}
$$

The new statevector becomes:

$$U_{hidden} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.44}$$

Firstly, let's define:

$$\begin{aligned} C_1 &= \cos(\frac{\theta_1+x_1}{2}), \quad S_1 = \sin(\frac{\theta_1+x_1}{2}) \\ C_2 &= \cos(\frac{\theta_2+x_2}{2}), \quad S_2 = \sin(\frac{\theta_2+x_2}{2}) \\ C_3 &= \cos(\frac{\theta_3+\theta_6+x_3}{2}), \quad S_3 = \sin(\frac{\theta_3+\theta_6+x_3}{2}) \\ c_j &= \cos(\frac{\theta_j}{2}), \quad s_j = \sin(\frac{\theta_j}{2}) \end{aligned}$$

$$|q_2q_1q_0\rangle = \begin{bmatrix} C_3c_5(c_4C_2C_1 + is_4S_2S_1) + S_3s_5(ic_4S_2C_1 + s_4C_2S_1) \\ C_3c_5(-is_4C_2C_1 - c_4S_2S_1) + S_3s_5(s_4S_2C_1 + ic_4C_2S_1) \\ C_3s_5(-ic_4C_2C_1 + s_4S_2S_1) + S_3c_5(-c_4S_2C_1 + is_4C_2S_1) \\ C_3s_5(-s_4C_2C_1 + ic_4S_2S_1) + S_3c_5(is_4S_2C_1 - c_4C_2S_1) \\ S_3c_5(-ic_4C_2C_1 + s_4S_2S_1) + C_3s_5(-c_4S_2C_1 + is_4C_2S_1) \\ S_3c_5(-s_4C_2C_1 + ic_4S_2S_1) + C_3s_5(is_4S_2C_1 - c_4C_2S_1) \\ S_3s_5(-c_4C_2C_1 - is_4S_2S_1) + C_3c_5(-ic_4S_2C_1 - s_4C_2S_1) \\ S_3s_5(is_4C_2C_1 + c_4S_2S_1) + C_3c_5(-s_4S_2C_1 - ic_4C_2S_1) \end{bmatrix} \tag{3.45}$$

However, as the circuit depth increases, manually deriving the statevector quickly becomes computationally intractable. This naturally raises an important question: can we abstract these increasingly complex expressions into a more compact mathematical representation that allows us to analyze the QNN systematically?

Fortunately, the answer is yes! In fact, under the data-encoding schemes considered here, the output of a QNN can be represented as a **finite Fourier series** [35].

To understand why, let us revisit Eq. 3.45. A closer examination reveals that each probability amplitude consists of combinations of sine and cosine functions. By applying standard trigonometric identities, these expressions can be rewritten in the general form

$$a_j(x,\theta) = \sum_{\alpha\in\Lambda} [A_{j,\alpha}(\theta)\cos(\alpha x) + B_{j,\alpha}(\theta)\sin(\alpha x)] \tag{3.46}$$

where Λ denotes a finite set of frequencies, and $A_{j,\alpha}(\theta)$ and $B_{j,\alpha}(\theta)$ are coefficients determined by the trainable parameters.

This is precisely the structure of a finite trigonometric series! More importantly, since the QNN output is obtained through the expectation value of an observable, it can also be expressed as a finite Fourier series:

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta) e^{i\omega x} \tag{3.47}$$

Here, Ω denotes the set of accessible frequencies. Unlike classical neural networks with learnable frequencies, such as SIREN [38], the accessible frequency spectrum of a QNN is primarily determined by its data-encoding strategy and circuit architecture. In particular, both circuit depth, characterized by the number of data-reuploading layers, and circuit width, characterized by the number of qubits involved in data encoding, can influence the accessible frequency spectrum [35]. We will explore these relationships in greater detail in the following sections.

Furthermore, $\omega \in \Omega$ represents an individual frequency component, while $c_\omega(\theta)$ denotes its corresponding trainable Fourier coefficient. From Fourier analysis, we know that the expressive power of a Fourier series—namely, the range of functions it can approximate—depends in part on the richness of its accessible frequency spectrum. Unlike in SIREN, where frequencies can be learned during training, the frequency spectrum of a QNN with a fixed encoding scheme is predetermined by its architecture rather than directly optimized through training.

This naturally raises an intriguing question: **Is a predetermined frequency spectrum a fundamental limitation of QNNs, or could it actually become an advantage?** We will investigate this question in the following sections.

This remarkable connection allows us to study the expressive power of QNNs through their accessible frequency spectra and trainable Fourier coefficients, rather than explicitly tracking the exponentially growing quantum statevector.

3.7 Output Layer: Quantum Measurement

The final stage of a QNN is the measurement layer, which converts quantum information into classical outputs. Unlike classical neural networks, where intermediate activations can be directly accessed, the quantum statesevolving throughout a quantum circuit cannot be directly observed without performing measurements. In particular, a computational-basis measurement of a qubit yields either 0 or 1, with probabilities governed by the Born rule [17, Sec. 2.2]. However, the complex probability amplitudes underlying these outcomes cannot be directly recovered from a single measurement.

How, then, can we extract meaningful numerical outputs from a quantum circuit? A common approach is to measure the expectation value of a quantum observable $\mathcal{O}$.

After the encoding layer and multiple variational rotation and entangling layers, the quantum state can be expressed as

$$|\psi(x,\theta)\rangle = U(x,\theta)\,|0\rangle^{\otimes n} \tag{3.48}$$

where $U(x, \theta)$ represents the overall unitary transformation of the circuit. Despite the potentially complex sequence of quantum operations, the entire ideal circuit can be mathematically represented by a single unitary operator.

Given an observable $\mathcal{O}$, the output of the QNN is defined as its expectation value:

$$\begin{aligned} f_\theta(x) &= \langle \psi(x,\theta) | \, \mathcal{O} \, | \psi(x,\theta) \rangle \\ &= \langle 0 |^{\otimes n} \, U^\dagger(x,\theta) \mathcal{O} U(x,\theta) \, | 0 \rangle^{\otimes n} \end{aligned} \tag{3.49}$$

One of the most commonly used observables in variational QNNs is the Pauli-Z operator:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \tag{3.50}$$

To understand how this measurement works, consider a single qubit in the state

$$|\psi\rangle = \alpha \, |0\rangle + \beta \, |1\rangle \, , \qquad |\alpha|^2 + |\beta|^2 = 1 \tag{3.51}$$

According to the Born rule, the probabilities of measuring the qubit in states $|0\rangle$ and $|1\rangle$ are

$$P(0) = |\alpha|^2, \qquad P(1) = |\beta|^2$$

Since the eigenvalues of the Pauli-Z operator are $+1$ and -1, its expectation value is

$$\begin{aligned} \langle Z \rangle &= \langle \psi | \, Z \, | \psi \rangle \\ &= |\alpha|^2 - |\beta|^2 \\ &= P(0) - P(1) \\ &= 2P(0) - 1 \end{aligned} \tag{3.52}$$

Consequently, the measurement probabilities can be recovered from the expectation value:

$$P(0) = \frac{1 + \langle Z \rangle}{2}, \qquad P(1) = \frac{1 - \langle Z \rangle}{2} \tag{3.53}$$

Since $-1 \leq \langle Z \rangle \leq 1$, an expectation value closer to $+1$ indicates a higher probability of measuring $|0\rangle$, whereas a value closer to -1 indicates a higher probability of measuring $|1\rangle$.

Importantly, the expectation value should not be confused with the probability amplitude. While the probability amplitudes α and β are generally complex-valued, the expectation value $\langle Z \rangle$ is real-valued and reflects the difference between the two measurement probabilities.

On real quantum hardware, the expectation value cannot be obtained from a single circuit execution. Instead, the circuit must be executed repeatedly to estimate the expectation value statistically [25, 26]:

$$\widehat{\langle Z \rangle} = \frac{N_0 - N_1}{N} \tag{3.54}$$

where N_0 and N_1 denote the numbers of measurement outcomes 0 and 1, respectively, and $N = N_0 + N_1$ is the total number of measurement shots.

For a multi-qubit system, the same principle applies. Consider the three-qubit QNN introduced earlier. Following the little-endian convention, its final statevector is

$$|\psi\rangle = \begin{bmatrix} a_{000} \\ a_{001} \\ a_{010} \\ a_{011} \\ a_{100} \\ a_{101} \\ a_{110} \\ a_{111} \end{bmatrix} \tag{3.55}$$

where

$$\sum_{i,j,k\in\{0,1\}} |a_{ijk}|^2 = 1 \tag{3.56}$$

To measure the first qubit, q_1, we define the observable

$$\mathcal{O}_1 = I \otimes I \otimes Z \tag{3.57}$$

The corresponding expectation value becomes

$$\begin{aligned} \langle Z_1 \rangle &= \langle \psi | \, \mathcal{O}_1 \, | \psi \rangle \\ &= |a_{000}|^2 - |a_{001}|^2 \\ &\quad + |a_{010}|^2 - |a_{011}|^2 \\ &\quad + |a_{100}|^2 - |a_{101}|^2 \\ &\quad + |a_{110}|^2 - |a_{111}|^2 \end{aligned} \tag{3.58}$$

This expression illustrates that measuring a single qubit generally involves contributions from multiple computational basis states. In particular, the expectation value depends on the marginal measurement probabilities associated with the selected qubit, rather than on any individual probability amplitude.

Similarly, the remaining two qubits can be measured using the observables

$$\mathcal{O}_2 = I \otimes Z \otimes I, \qquad \mathcal{O}_3 = Z \otimes I \otimes I \tag{3.59}$$

By measuring all three qubits, the QNN produces a vector of expectation values:

$$\mathbf{f}_\theta(x) = \begin{bmatrix} \langle Z_1 \rangle \\ \langle Z_2 \rangle \\ \langle Z_3 \rangle \end{bmatrix} \tag{3.60}$$

These expectation values serve as classical output features and can either be used directly for prediction or passed to an additional classical processing layer.

Beyond Pauli-Z measurements, other observables, such as Pauli-X, Pauli-Y, and multi-qubit correlation operators, can also be employed to extract different properties of the quantum state. However, a detailed discussion of these measurement strategies is beyond the scope of this section.

In summary, quantum measurement provides the essential bridge between the quantum representation and the classical output of a QNN. More importantly, expressing the QNN

output as the expectation value of an observable establishes the mathematical foundation for analyzing its function representation, particularly through Fourier analysis.

4 Fourier Analysis of Quanutm Neural Networks and Expressivity

4.1 Basics of Fourier Series and Fourier Transform

Fourier analysis provides a powerful mathematical framework for representing and analyzing functions in terms of their frequency components. Standard treatments cover both Fourier series and Fourier transforms [43]. The fundamental idea is that a function can be decomposed into a combination of oscillatory basis functions, such as sines, cosines, or complex exponentials. Instead of examining a function solely in its original domain, Fourier analysis allows us to study its behavior in the frequency domain.

Two fundamental tools in Fourier analysis are the *Fourier series* and the *Fourier transform*. Although closely related, they serve different purposes and apply to different classes of functions.

Fourier Series

A Fourier series represents a periodic function as an infinite sum of sinusoidal components with different frequencies, amplitudes, and phases.

For a periodic function $f(x)$ with period T, its Fourier series can be expressed in complex exponential form as

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ik\omega_0 x} \tag{4.1}$$

where $\omega_0 = 2\pi/T$ is the fundamental angular frequency, $k \in \mathbb{Z}$ is the harmonic index, and c_k denotes the Fourier coefficient associated with frequency $k\omega_0$.

Each Fourier coefficient can be calculated by projecting the original function onto its corresponding complex exponential basis:

$$c_k = \frac{1}{T} \int_0^T f(x) e^{-ik\omega_0 x}\, dx \tag{4.2}$$

These coefficients describe the contribution of each frequency component to the original function. In particular, $|c_k|$ determines its magnitude, while the complex phase of c_k determines its phase shift.

According to Euler's formula,

$$e^{i\omega x} = \cos(\omega x) + i \sin(\omega x) \tag{4.3}$$

the complex exponential representation is equivalent to expressing a real-valued periodic function as a sum of sine and cosine functions.

Importantly, different frequencies capture different levels of variation in a function. Low-frequency components describe slowly varying patterns, whereas high-frequency components represent rapid oscillations and fine-grained details.

In practical applications, Fourier series are often approximated using a finite number of frequency components. More generally, a finite trigonometric expansion can be written as

$$f(x) = \sum_{\omega \in \Omega} c_\omega e^{i\omega x} \tag{4.4}$$

where Ω is a finite set of frequencies. Unlike the conventional Fourier series, thefrequencies in this generalized representation are not necessarily integer multiples of a common fundamental frequency.

Fourier Transform

While Fourier series decompose periodic functions into discrete harmonics, the Fourier transform extends frequency-domain analysis to nonperiodic functions.

For a sufficiently well-behaved function $f(x)$, the continuous Fourier transform is defined as

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(x) e^{-i\omega x}\, dx \tag{4.5}$$

where $\omega \in \mathbb{R}$ represents the angular frequency and $\hat{f}(\omega)$ describes the frequency-domain representation of $f(x)$.

The original function can be reconstructed using the inverse Fourier transform:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega) e^{i\omega x}\, d\omega \tag{4.6}$$

The key distinction is that the Fourier series uses a discrete set of frequencies, whereas the Fourier transform is defined over a continuous frequency variable. Nevertheless, both share the same fundamental principle: a function can be understood through its frequency components and their corresponding coefficients.

From a function-approximation perspective, the frequency spectrum reveals which oscillatory patterns are present in a function. This perspective is particularly useful when studying the expressive power of machine learning models.

4.2 Connection to Quantum Neural Networks

The significance of Fourier analysis becomes apparent when examining the mathematical representation of Quantum Neural Networks (QNNs).

As discussed previously, the output of a variational QNN is typically defined through the expectation value of an observable:

$$f_\theta(x) = \langle 0 | U^\dagger(x, \theta) \mathcal{O} U(x, \theta) | 0 \rangle \tag{4.7}$$

For the finite-dimensional quantum circuits with data-encoding operations considered in this paper, this expectation-value function can be expressed as a finite Fourier series [35]:

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta) e^{i\omega x} \tag{4.8}$$

Here, Ω denotes the set of accessible frequencies, while $c_\omega(\theta)$ represents the Fourier coefficients controlled by the trainable circuit parameters.

Unlike conventional Fourier approximations, where the selected frequencies may be chosen or optimized for a particular task, the accessible frequencies of a QNN with a fixed data-encoding scheme are determined by its encoding operations and circuit architecture. The trainable parameters primarily control the corresponding Fourier coefficients, subject to constraints imposed by the circuit [35].

Therefore, Fourier analysis provides a natural framework for investigating two fundamental aspects of QNN expressivity: which frequencies a quantum circuit can represent, and how flexibly it can adjust their coefficients.

In the following section, we will examine how these properties emerge from quantum data encoding and how circuit depth and width influence the accessible frequency spectrum.

4.3 Quantum Neural Networks as Finite Fourier Series

This section presents a detailed derivation of a result due to Schuld, Sweke, and Meyer [35]: the output of a quantum neural network (QNN) in which the input is encoded through Hamiltonian-generated gates can be expressed as a finite Fourier series in that input. The argument proceeds in four stages. First, we fix the general form of the circuit and show that the encoding gates may, without loss of generality, be taken to be diagonal. Second, we compute the state vector for one, two, and finally L encoding layers, and show that every amplitude is a sum of complex exponentials whose frequencies are sums of eigenvalues of the encoding Hamiltonian. Third, we evaluate the measured expectation value and show that the frequencies of the model output are *differences* of these eigenvalue sums. Finally, we collect terms of equal frequency to obtain the Fourier representation and illustrate it with Pauli-rotation encoding.

Conventions. We consider a scalar input $x \in \mathbb{R}$ and a circuit on N qubits, so that the Hilbert space has dimension $d = 2^N$. Computational basis states are indexed by $1, \ldots, d$, with the initial state $|0\rangle \equiv |0\rangle^{\otimes N}$ corresponding to the first basis vector. For a matrix W, W_{mn} denotes the entry in row m and column n. The symbol i is reserved for the imaginary unit; matrix and vector indices are denoted by m, n, j, and k.

Circuit model. The quantum circuits considered here share the general structure shown in Figure 11, in which data-independent blocks alternate with data-encoding gates. The overall unitary is therefore

$$U(x) = W^{(L+1)} S(x)\, W^{(L)} \cdots W^{(2)} S(x)\, W^{(1)}, \tag{4.9}$$

where each $W^{(l)}$ is a unitary that does not depend on x. These blocks may contain trainable parameterized gates, such as $R_X(\theta)$, as well as fixed gates, such as the Hadamard gate. The gate $S(x)$ encodes the classical input x into the quantum state and is repeated L times. We assume that $S(x)$ is generated by a Hermitian operator H, referred to as the *encoding Hamiltonian*:

$$S(x) = e^{-ixH}. \tag{4.10}$$

For example, encoding with the single-qubit rotation $R_X(x)$ corresponds to

$$H = \frac{1}{2} X = \frac{1}{2} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{4.11}$$

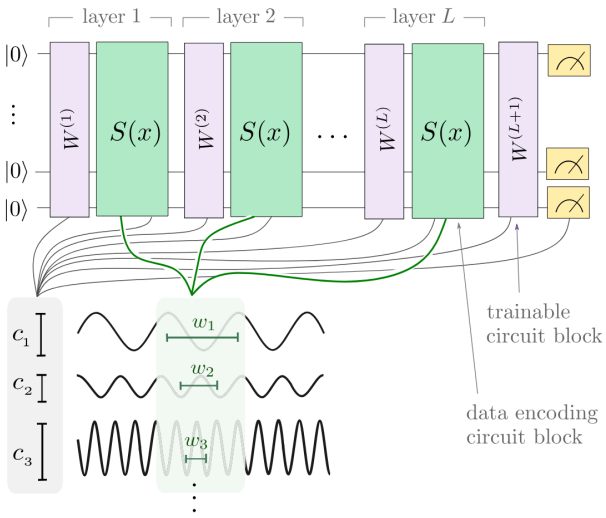


Figure 11: General structure of the quantum circuits considered in this section: data-independent blocks $W^{(l)}$ alternate with data-encoding gates $S(x)$.

Indeed, since $X^2 = I$, the Taylor series of the exponential separates into even powers, which are proportional to I, and odd powers, which are proportional to X:

$$\begin{aligned} e^{-i\frac{x}{2}X} &= \sum_{n=0}^{\infty} \frac{1}{n!} \left(-\frac{ix}{2}\right)^n X^n \\ &= \left[\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k)!} \left(\frac{x}{2}\right)^{2k}\right] I - i \left[\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} \left(\frac{x}{2}\right)^{2k+1}\right] X \\ &= \cos\left(\frac{x}{2}\right) I - i \sin\left(\frac{x}{2}\right) X = R_X(x). \end{aligned} \tag{4.12}$$

The output of the model is the expectation value of a Hermitian observable M (which need not be unitary) with respect to the final state:

$$f(x) = \langle \psi(x) | \, M \, | \psi(x) \rangle \,, \qquad |\psi(x)\rangle = U(x) \, |0\rangle \,. \tag{4.13}$$

The trainable parameters θ enter only through the blocks $W^{(l)}$; for brevity, we suppress the dependence on θ throughout.

Step 1: Diagonalizing the encoding gate. Because H is Hermitian, the spectral theorem guarantees the decomposition

$$H = V^\dagger \Sigma V, \qquad \Sigma = \mathrm{diag}(\lambda_1, \ldots, \lambda_d), \tag{4.14}$$

where V is unitary and the eigenvalues $\lambda_1, \ldots, \lambda_d$ are real. Since V is unitary, $(V^\dagger \Sigma V)^n = V^\dagger \Sigma^n V$ for every n, and the power-series definition of the matrix exponential gives

$$S(x) = V^\dagger e^{-ix\Sigma} V, \qquad e^{-ix\Sigma} = \mathrm{diag}\left(e^{-ix\lambda_1}, \ldots, e^{-ix\lambda_d}\right). \tag{4.15}$$

Substituting this expression into Eq. (4.9), every factor V or $V^\dagger$ is adjacent to a data-independent block:

$$U(x) = W^{(L+1)}V^\dagger e^{-ix\Sigma}V\,W^{(L)}V^\dagger e^{-ix\Sigma}V\cdots V\,W^{(2)}V^\dagger e^{-ix\Sigma}V\,W^{(1)}. \tag{4.16}$$

These factors can therefore be absorbed into the neighbouring blocks by defining

$$\widetilde{W}^{(1)} = VW^{(1)}, \qquad \widetilde{W}^{(l)} = VW^{(l)}V^\dagger \;\; (2 \le l \le L), \qquad \widetilde{W}^{(L+1)} = W^{(L+1)}V^\dagger, \tag{4.17}$$

which yields

$$U(x) = \widetilde{W}^{(L+1)}e^{-ix\Sigma}\,\widetilde{W}^{(L)}\cdots\widetilde{W}^{(2)}e^{-ix\Sigma}\,\widetilde{W}^{(1)}. \tag{4.18}$$

The new blocks $\widetilde{W}^{(l)}$ are still unitary and independent of x. Consequently, we may assume without loss of generality that the encoding gate is diagonal in the computational basis. To simplify notation, we drop the tildes and write from now on

$$S(x) = \mathrm{diag}\Big(e^{-ix\lambda_1},\ldots,e^{-ix\lambda_d}\Big). \tag{4.19}$$

Step 2: A single encoding layer ($L = 1$). We now compute the amplitudes of the final state, beginning with the simplest case,

$$U(x) = W^{(2)}S(x)\,W^{(1)}. \tag{4.20}$$

Because $|0\rangle$ is the first basis vector, the first block maps it to the first column of $W^{(1)}$:

$$|\phi\rangle = W^{(1)}|0\rangle = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_d \end{bmatrix}, \qquad a_j = W^{(1)}_{j1}. \tag{4.21}$$

Since $S(x)$ is diagonal, the encoding gate multiplies each component by a phase:

$$S(x)\,|\phi\rangle = \begin{bmatrix} a_1e^{-ix\lambda_1} \\ a_2e^{-ix\lambda_2} \\ \vdots \\ a_de^{-ix\lambda_d} \end{bmatrix}. \tag{4.22}$$

Applying the second block $W^{(2)}$, the m-th component of the final state is

$$\begin{aligned} \psi_m(x) &= W^{(2)}_{m1}a_1e^{-ix\lambda_1} + W^{(2)}_{m2}a_2e^{-ix\lambda_2} + \cdots + W^{(2)}_{md}a_de^{-ix\lambda_d} \\ &= \sum_{j=1}^{d} W^{(2)}_{mj}W^{(1)}_{j1}\,e^{-ix\lambda_j}. \end{aligned} \tag{4.23}$$

Each amplitude is thus a linear combination of the exponentials $e^{-ix\lambda_j}$, with coefficients that depend only on the data-independent blocks.

Step 3: Two encoding layers ($L = 2$). Next, consider

$$U(x) = W^{(3)} S(x)\, W^{(2)} S(x)\, W^{(1)}. \tag{4.24}$$

Let $\phi^{(l)}$ denote the state immediately after block $W^{(l)}$, and $\hat{\phi}^{(l)}$ the state after the encoding gate that follows it. The indices $j_1, j_2 \in \{1, \ldots, d\}$ are independent. After the first block,

$$\phi^{(1)}_{j_1} = W^{(1)}_{j_1 1}, \tag{4.25}$$

and after the first encoding gate,

$$\hat{\phi}^{(1)}_{j_1} = e^{-ix\lambda_{j_1}} W^{(1)}_{j_1 1}. \tag{4.26}$$

Applying the second block gives

$$\phi^{(2)}_{j_2} = \sum_{j_1=1}^{d} W^{(2)}_{j_2 j_1} e^{-ix\lambda_{j_1}} W^{(1)}_{j_1 1}. \tag{4.27}$$

The sum arises because the j_2-th row of $W^{(2)}$ combines all d components of the preceding state. The second encoding gate multiplies the j_2-th component by $e^{-ix\lambda_{j_2}}$. Since $e^{-ix\lambda_{j_2}} e^{-ix\lambda_{j_1}} = e^{-ix(\lambda_{j_1}+\lambda_{j_2})}$, we obtain

$$\hat{\phi}^{(2)}_{j_2} = e^{-ix\lambda_{j_2}} \phi^{(2)}_{j_2} = \sum_{j_1=1}^{d} W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}\, e^{-ix(\lambda_{j_1}+\lambda_{j_2})}. \tag{4.28}$$

Finally, applying the third block yields

$$\psi_m(x) = \sum_{j_2=1}^{d} W^{(3)}_{m j_2} \hat{\phi}^{(2)}_{j_2} = \sum_{j_1=1}^{d} \sum_{j_2=1}^{d} W^{(3)}_{m j_2} W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}\, e^{-ix(\lambda_{j_1}+\lambda_{j_2})}. \tag{4.29}$$

Each term factorizes into two parts:

1. an input-independent coefficient, $W^{(3)}_{m j_2} W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}$, determined by the data-independent blocks; and
2. an input-dependent phase, $e^{-ix(\lambda_{j_1}+\lambda_{j_2})}$, whose frequency is a sum of eigenvalues of the encoding Hamiltonian, one from each encoding layer.

Step 4: General number of encoding layers. The same pattern persists for any number of layers: each additional encoding gate contributes one phase factor, and each additional block introduces one summation index. Formally, the result follows by induction on L. If the components after block $W^{(l)}$ are sums over $j_1, \ldots, j_{l-1}$ of products of block entries multiplied by $e^{-ix(\lambda_{j_1}+\cdots+\lambda_{j_{l-1}})}$, then the subsequent encoding gate multiplies component j_l by $e^{-ix\lambda_{j_l}}$, and block $W^{(l+1)}$ introduces a sum over j_l; this reproduces the same form with l replaced by $l+1$. For L encoding layers we therefore have

$$\psi_m(x) = \sum_{j_1,\ldots,j_L=1}^{d} A_{m,j_1,\ldots,j_L}\, e^{-ix(\lambda_{j_1}+\cdots+\lambda_{j_L})}, \tag{4.30}$$

where the coefficients are products of entries of the data-independent blocks:

$$A_{m,j_1,\ldots,j_L} = W^{(L+1)}_{mj_L} W^{(L)}_{j_L j_{L-1}} \cdots W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}. \tag{4.31}$$

To shorten the notation, we collect the indices into a multi-index and define the corresponding eigenvalue sum:

$$\mathbf{j} = (j_1, \ldots, j_L) \in [d]^L, \qquad \Lambda_{\mathbf{j}} = \lambda_{j_1} + \lambda_{j_2} + \cdots + \lambda_{j_L}, \tag{4.32}$$

where $[d] = \{1, \ldots, d\}$, so that $[d]^L$ is the set of all d^L index tuples of length L. The amplitudes then take the compact form

$$\psi_m(x) = \sum_{\mathbf{j} \in [d]^L} A_{m,\mathbf{j}}\, e^{-ix\Lambda_{\mathbf{j}}}. \tag{4.33}$$

In words, regardless of the depth of the circuit, every amplitude is a sum of terms of the form (circuit coefficient) $\times\, e^{-ix\cdot(\text{eigenvalue sum})}$. The coefficients depend on the trainable blocks, whereas the eigenvalue sums depend only on the encoding Hamiltonian and on L.

For example, with $d = 2$ and $L = 2$, the multi-index ranges over

$$\mathbf{j} \in \{(1,1), (1,2), (2,1), (2,2)\}, \tag{4.34}$$

and Eq. (4.33) contains four terms:

$$\psi_m(x) = A_{m,(1,1)} e^{-2ix\lambda_1} + A_{m,(1,2)} e^{-ix(\lambda_1+\lambda_2)} + A_{m,(2,1)} e^{-ix(\lambda_2+\lambda_1)} + A_{m,(2,2)} e^{-2ix\lambda_2}. \tag{4.35}$$

The multi-indices $(1,2)$ and $(2,1)$ yield the same eigenvalue sum, so the corresponding terms share a frequency. Such coincidences are common and will be used in Step 6 to combine terms.

Step 5: Measurement. The amplitudes contain frequencies $\Lambda_{\mathbf{j}}$, whereas the frequencies of the model output turn out to be differences $\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}}$. This change originates in the measurement: the expectation value in Eq. (4.13) is quadratic in the state and involves both the amplitudes and their complex conjugates. Writing

$$|\psi(x)\rangle = \begin{bmatrix} \psi_1(x) \\ \vdots \\ \psi_d(x) \end{bmatrix}, \qquad \langle\psi(x)| = \begin{bmatrix} \psi_1^*(x) & \cdots & \psi_d^*(x) \end{bmatrix}, \tag{4.36}$$

the expectation value becomes

$$f(x) = \sum_{m=1}^{d} \sum_{n=1}^{d} \psi_m^*(x)\, M_{mn}\, \psi_n(x). \tag{4.37}$$

For a single qubit ($d = 2$), for instance,

$$\begin{aligned} f(x) &= \begin{bmatrix} \psi_1^* & \psi_2^* \end{bmatrix} \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \begin{bmatrix} \psi_1 \\ \psi_2 \end{bmatrix} \\ &= \psi_1^* M_{11} \psi_1 + \psi_1^* M_{12} \psi_2 + \psi_2^* M_{21} \psi_1 + \psi_2^* M_{22} \psi_2. \end{aligned} \tag{4.38}$$

The off-diagonal entries of M couple different amplitudes through cross terms. Even when M is diagonal (e.g., $M = Z$), the output contains the products $\psi_m^* \psi_m$ and therefore remains quadratic in the amplitudes.

To evaluate Eq. (4.37), we require the complex conjugate of Eq. (4.33). Because the eigenvalue sums $\Lambda_{\mathbf{k}}$ are real,

$$\psi_m^*(x) = \sum_{\mathbf{k} \in [d]^L} A_{m,\mathbf{k}}^* \, e^{ix\Lambda_{\mathbf{k}}}. \tag{4.39}$$

Conjugation has two effects: each coefficient is replaced by its complex conjugate, and the sign of the exponent changes from negative to positive. We also rename the summation multi-index to $\mathbf{k}$; it ranges over the same set $[d]^L$ as $\mathbf{j}$, but the two sums are independent. Substituting both expansions into Eq. (4.37) gives

$$\begin{aligned} f(x) &= \sum_{m,n} \left(\sum_{\mathbf{k}} A_{m,\mathbf{k}}^* \, e^{ix\Lambda_{\mathbf{k}}} \right) M_{mn} \left(\sum_{\mathbf{j}} A_{n,\mathbf{j}} \, e^{-ix\Lambda_{\mathbf{j}}} \right) \\ &= \sum_{m,n} \sum_{\mathbf{k},\mathbf{j}} A_{m,\mathbf{k}}^* \, M_{mn} \, A_{n,\mathbf{j}} \, e^{ix\Lambda_{\mathbf{k}}} e^{-ix\Lambda_{\mathbf{j}}}. \end{aligned} \tag{4.40}$$

Using $e^{ix\Lambda_{\mathbf{k}}} e^{-ix\Lambda_{\mathbf{j}}} = e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}$ and exchanging the order of summation, we obtain

$$f(x) = \sum_{\mathbf{k},\mathbf{j} \in [d]^L} \left(\sum_{m,n} A_{m,\mathbf{k}}^* \, M_{mn} \, A_{n,\mathbf{j}} \right) e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}. \tag{4.41}$$

Defining the input-independent coefficients

$$a_{\mathbf{k},\mathbf{j}} = \sum_{m,n} A_{m,\mathbf{k}}^* \, M_{mn} \, A_{n,\mathbf{j}}, \tag{4.42}$$

the model output becomes

$$f(x) = \sum_{\mathbf{k},\mathbf{j} \in [d]^L} a_{\mathbf{k},\mathbf{j}} \, e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}. \tag{4.43}$$

Thus the output is a sum of terms of the form (coefficient) $\times \, e^{ix \cdot (\text{difference of eigenvalue sums})}$.

Step 6: Frequency spectrum and Fourier coefficients. Comparing each term in Eq. (4.43) with the Fourier basis function $e^{i\omega x}$, we identify its frequency as

$$\omega = \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \sum_{l=1}^{L} \lambda_{k_l} - \sum_{l=1}^{L} \lambda_{j_l}. \tag{4.44}$$

The set of frequencies accessible to the model, its *frequency spectrum*, is therefore

$$\Omega = \left\{ \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} \ : \ \mathbf{k}, \mathbf{j} \in [d]^L \right\}. \tag{4.45}$$

To summarize the notation: λ denotes an eigenvalue of the encoding Hamiltonian, Λ a sum of eigenvalues across the L encoding layers, and ω a difference of two such sums.

As anticipated in Step 4, distinct pairs of multi-indices may produce the same frequency. If, for example, $\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \Lambda_{\mathbf{k}'} - \Lambda_{\mathbf{j}'} = \omega$ for two different pairs $(\mathbf{k}, \mathbf{j})$ and $(\mathbf{k}', \mathbf{j}')$, the corresponding terms combine as

$$a_{\mathbf{k},\mathbf{j}}\, e^{i\omega x} + a_{\mathbf{k}',\mathbf{j}'}\, e^{i\omega x} = (a_{\mathbf{k},\mathbf{j}} + a_{\mathbf{k}',\mathbf{j}'})\, e^{i\omega x}. \tag{4.46}$$

Collecting all pairs that share a frequency, we define the Fourier coefficient associated with $\omega \in \Omega$ as

$$c_\omega = \sum_{\substack{\mathbf{k},\mathbf{j} \in [d]^L \\ \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \omega}} a_{\mathbf{k},\mathbf{j}}, \tag{4.47}$$

and arrive at the Fourier representation of the QNN:

$$\boxed{f(x) = \sum_{\omega \in \Omega} c_\omega\, e^{i\omega x}} \tag{4.48}$$

Since Ω contains at most d^{2L} elements, this is a finite sum. When the eigenvalue differences are commensurate, as for the Pauli encodings discussed below, all frequencies are integer multiples of a fundamental frequency and Eq. (4.48) is a truncated Fourier series in the usual sense; in general, it is a finite sum of complex exponentials.

The representation has two further properties. First, the spectrum is symmetric and contains zero: if $\omega = \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} \in \Omega$, then $-\omega = \Lambda_{\mathbf{j}} - \Lambda_{\mathbf{k}} \in \Omega$, and $\mathbf{k} = \mathbf{j}$ gives $\omega = 0$. Second, because M is Hermitian ($M^*_{mn} = M_{nm}$), Eq. (4.42) implies

$$a^*_{\mathbf{k},\mathbf{j}} = \sum_{m,n} A_{m,\mathbf{k}}\, M^*_{mn}\, A^*_{n,\mathbf{j}} = \sum_{m,n} A^*_{n,\mathbf{j}}\, M_{nm}\, A_{m,\mathbf{k}} = a_{\mathbf{j},\mathbf{k}}, \tag{4.49}$$

and summing over all pairs with frequency ω gives $c_{-\omega} = c^*_\omega$. Consequently, $f(x)$ is real-valued, as an expectation value of a Hermitian observable must be, and can equivalently be written as

$$f(x) = c_0 + \sum_{\omega \in \Omega,\, \omega > 0} 2\,|c_\omega| \cos(\omega x + \arg c_\omega)\,. \tag{4.50}$$

Roles of the encoding and trainable blocks. The blocks $W^{(l)}$ do not appear in the frequency spectrum (4.45), which is determined entirely by the eigenvalues of the encoding Hamiltonian and the number of encoding layers L. This is because the blocks are independent of x: they enter only through the coefficients $A_{m,\mathbf{j}}$ in Eq. (4.31), and hence only through $a_{\mathbf{k},\mathbf{j}}$ and c_ω, while leaving the exponential factors $e^{i\omega x}$ unchanged. Training therefore adjusts the amplitudes and phases of a fixed set of frequencies but cannot introduce frequencies outside Ω. The extent to which the coefficients themselves can be controlled depends on the expressivity of the trainable blocks. If different encoding Hamiltonians $H^{(l)}$ are used in different layers, the same derivation applies with $\Lambda_{\mathbf{j}} = \sum_l \lambda^{(l)}_{j_l}$, where $\lambda^{(l)}_j$ are the eigenvalues of $H^{(l)}$.

Example: Pauli-rotation encoding. For single-qubit encoding with $R_X(x)$, we have $H = \frac{1}{2}X$ with eigenvalues $\pm\frac{1}{2}$. Each eigenvalue sum $\Lambda_{\mathbf{j}}$ is a sum of L terms equal to $\pm\frac{1}{2}$ and

therefore takes values in $\{-\frac{L}{2}, -\frac{L}{2}+1, \ldots, \frac{L}{2}\}$. Their differences are the integers from $-L$ to L, so that

$$\Omega = \{-L, \ldots, -1, 0, 1, \ldots, L\}, \qquad f(x) = \sum_{\omega=-L}^{L} c_\omega \, e^{i\omega x}. \tag{4.51}$$

A model that encodes x L times through a single-qubit Pauli rotation is thus a truncated Fourier series of degree L. In particular, with a single encoding layer ($L = 1$), the model reduces to $f(x) = c_0 + 2|c_1| \cos(x + \arg c_1)$, a single sinusoid, irrespective of how expressive the trainable blocks are. Accessing higher frequencies requires encoding the input repeatedly, either sequentially (data re-uploading, larger L) or in parallel; for instance, applying $R_X(x)$ simultaneously to N qubits gives $H = \frac{1}{2}\sum_{q=1}^{N} X_q$, whose eigenvalues range over $\{-\frac{N}{2}, \ldots, \frac{N}{2}\}$, so a single such layer already yields the spectrum $\{-N, \ldots, N\}$.

4.4 Expressivity and Frequency Expansion

Now, we have seen that the frequency component QNN can learn depends on the encoding Hamiltonian. However, the purpose of deep learning is to approximate any functions. For an arbitrary function, the frequency components are also arbitrary. Then, we need to expand and enrich the frequency spectrum QNN can learn. In another word, we must make sure QNN can learn an arbitrary combination of frequency independent in order to make it an Universal Function Approximator. The question is how do we do it? There are 2 ways: (1) Encoding inputs using more qubits and (2) Encoding the input more times on the same qubits. Let's see how it works in the following.

4.4.1 Increasing Encoding Qubits

As established above, the frequencies accessible to the model are determined by the differences between the eigenvalues of the encoding Hamiltonian. Consequently, the frequency spectrum can be enlarged either by increasing the number of qubits on which the input is encoded (the width) or by increasing the number of times the input is re-uploaded (the depth). The following example illustrates the former.

Example. Consider first a single encoding qubit. The encoding gate is given by

$$S_1(x) = e^{-ixH}, \qquad H = \frac{1}{2}X = \frac{1}{2}\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \tag{4.52}$$

where X denotes the Pauli-X operator. The eigenvalues of H are $\lambda(H) = \{-\frac{1}{2}, \frac{1}{2}\}$, and the resulting frequency spectrum, obtained from all pairwise eigenvalue differences, is $\Omega = \{-1, 0, 1\}$. If the number of encoding qubits is increased to two, with $H_1 = H_2 = \frac{1}{2}X$ acting on the respective qubits, the encoding becomes

$$S_2(x) = e^{-ixH_2} \otimes e^{-ixH_1} = e^{-ix(H_2 \otimes I + I \otimes H_1)}, \tag{4.53}$$

where the second equality holds because $H_2 \otimes I$ and $I \otimes H_1$ commute. The effective encoding Hamiltonian is therefore

$$H = \frac{1}{2}\left(X \otimes I + I \otimes X\right), \tag{4.54}$$

whose distinct eigenvalues are $\lambda(H) = \{-1, 0, 1\}$, yielding the frequency spectrum $\Omega = \{-2, -1, 0, 1, 2\}$. More generally, for N qubits, each encoded exactly once, the distinct eigenvalues of the encoding Hamiltonian are

$$\lambda_k(H) = \frac{N}{2} - k, \qquad k = 0, 1, \ldots, N, \tag{4.55}$$

and the corresponding frequency spectrum is

$$\Omega(H) = \{-N, -N+1, \ldots, -1, 0, 1, \ldots, N-1, N\}. \tag{4.56}$$

Hence, N encoding qubits give access to $2N + 1$ integer frequencies. Since the spectrum is symmetric about zero and the model output is real-valued, this corresponds to $N + 1$ independent non-negative frequencies, namely $\{0, 1, \ldots, N\}$.

An example is shown in Figure 13. The unscaled input x is encoded once Each trainable layer consists of parameterized single-qubit rotations R_X, R_Y, and R_Z, followed by a ring of CNOT gates that couples each qubit to its neighbor, with the last qubit connected back to the first, thereby generating entanglement across the register. Theoretically, the accessible frequencies are $\{0, \pm1, \pm2, \pm3, \pm4, \pm5\}$.

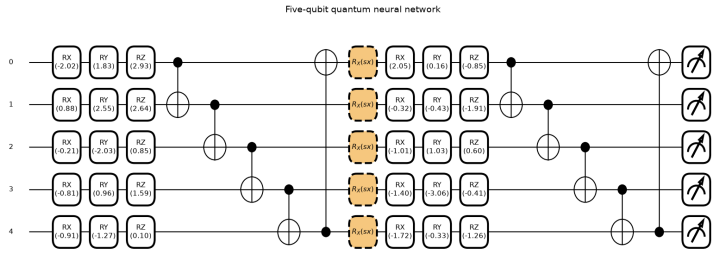


Figure 12: Architecture of the five-qubit QNN, consisting of a single encoding layer followed by two layers of parameterized rotation gates, each followed by a CNOT ring.

4.4.2 Data-Reuploading

Now, a natural question rises. If we are limited on the budget of qubits, and only has 1, can the circuit still learn multiple frequencies? The answer is Yes and the approach is called **Data-Reuploading**, which simply is encoding the same x multiple times on a single qubit.

$$U(x) = W(\theta_1)S^1(x)W(\theta_2)S^2(x)W(\theta_3)S(x)\cdots W(\theta_L)S^L(x)M \tag{4.57}$$

Let's say we encode x twice on a qubit, then we have 2 encoding hamiltonians as follows

$$
\begin{aligned}
H_1 = H_2 = \frac{1}{2}X &= \frac{1}{2}\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \\
\operatorname{spec}(H_1) = \operatorname{spec}(H_2) &= \left\{-\frac{1}{2}, \frac{1}{2}\right\}, \\
\Lambda &= \left\{\lambda_{j_1}^{(1)} + \lambda_{j_2}^{(2)} \;\middle|\; j_1, j_2 \in \{0,1\}\right\} = \{-1, 0, 1\}, \\
\Omega &= \left\{\sum_{\ell=1}^{2}\left(\lambda_{j_\ell}^{(\ell)} - \lambda_{k_\ell}^{(\ell)}\right) \;\middle|\; j_\ell, k_\ell \in \{0,1\}\right\} \\
&= \{\lambda - \lambda' \mid \lambda, \lambda' \in \Lambda\} \\
&= \{-2, -1, 0, 1, 2\}.
\end{aligned}
\tag{4.58}
$$

Where j, k are the independent indexes. If we generalize to L encodings as shown in Equation 4.79, we have

$$
H_{1,\cdots,L} = \{-\frac{1}{2}, \frac{1}{2}\} \tag{4.59}
$$

The set of all possible sums of eigenvalues across the L encoding layers is given by

$$
\begin{aligned}
\Lambda(H_1, \ldots, H_L) &= \left\{\sum_{l=1}^{L} \lambda_{j_l}^{(l)} \;\middle|\; \lambda_{j_l}^{(l)} \in \operatorname{spec}(H_l)\right\} \\
&= \left\{-\frac{L}{2}, -\frac{L}{2}+1, \ldots, \frac{L}{2}\right\},
\end{aligned}
\tag{4.60}
$$

Then, accessible frequencies are

$$
\Omega = \Lambda_J - \lambda_K = \{0, \pm 1, \pm 2, \cdots, \pm L\} \tag{4.61}
$$

4.4.3 Encoding High Frequency

So far have focused on target functions that contain only low frequencies. In many applications, however, high-frequency components must also be captured. Under the encoding scheme considered previously, in which x is encoded once on each of N qubits, the largest accessible frequency is N; a target containing the frequency $\omega = 1000$ would therefore require 1000 qubits. This requirement can be avoided by multiplying the input x by a constant coefficient before it is encoded.

Example. Consider a two-qubit QNN in which x is encoded once on each qubit. As shown previously, the accessible frequencies are $\{-2, -1, 0, 1, 2\}$. If the inputs to the two qubits are instead scaled by coefficients $a, b \in \mathbb{R}$, the encoding becomes

$$
S(x) = e^{-iaxH_1} \otimes e^{-ibxH_2} = e^{-ix(aH_1 \otimes I + I \otimes bH_2)}, \qquad H_1 = H_2 = \frac{1}{2}X, \tag{4.62}
$$

so that the effective encoding Hamiltonian is

$$
H = aH_1 \otimes I + I \otimes bH_2. \tag{4.63}
$$

Its eigenvalues are

$$
\lambda(H) = \left\{\frac{a+b}{2}, \;\frac{a-b}{2}, \;\frac{-a+b}{2}, \;\frac{-a-b}{2}\right\}, \tag{4.64}
$$

and the accessible frequencies are therefore

$$\Omega = \{0,\ \pm a,\ \pm b,\ \pm(a+b),\ \pm(a-b)\}\,. \tag{4.65}$$

High frequencies thus become accessible without increasing the number of qubits. For instance, the frequency $\omega = 1000$ is accessible if $a = 1000$, or if a and b are chosen such that $a+b = 1000$ or $a - b = 1000$.

The same argument extends to N qubits, where the j-th qubit encodes $a_j x$. The encoding Hamiltonian, its eigenvalues, and the resulting frequency spectrum are

$$\begin{aligned} H &= \frac{1}{2}\sum_{j=1}^{N} a_j X_j, \\ \lambda &= \frac{1}{2}\sum_{j=1}^{N} \epsilon_j a_j, \qquad \epsilon_j \in \{-1, 1\}, \\ \Omega &= \left\{ \sum_{j=1}^{N} m_j a_j \;\middle|\; m_j \in \{-1, 0, 1\} \right\}, \end{aligned} \tag{4.66}$$

where X_j denotes the Pauli-X operator acting on the j-th qubit. The scaled encoding used in the previous experiments corresponds to $N = 5$ and $a_j = j$, for which $\Omega = \{-15, \ldots, 15\}$.

High frequencies can also be accessed with a single qubit by re-uploading the input several times, with each encoding scaled by its own coefficient. For a single-qubit QNN with L re-uploading layers, the circuit is

$$U(x) = W(\theta_{L+1})\, S^{(L)}(x)\, W(\theta_L) \cdots W(\theta_2)\, S^{(1)}(x)\, W(\theta_1), \tag{4.67}$$

where each $W(\theta_l)$ is a trainable block, and the model output is $f(x) = \langle 0|U^\dagger(x)\, M\, U(x)|0\rangle$ for an observable M. The l-th encoding unitary is

$$S^{(l)}(x) = R_X(s_l x) = e^{-ixH_l}, \qquad H_l = \frac{s_l}{2} X, \tag{4.68}$$

where $s_l \in \mathbb{R}$ denotes the encoding coefficient of the l-th layer. The eigenvalues of H_l are

$$\mathrm{spec}(H_l) = \left\{\lambda_0^{(l)}, \lambda_1^{(l)}\right\} = \left\{-\frac{s_l}{2}, \frac{s_l}{2}\right\}. \tag{4.69}$$

The set of all sums of eigenvalues across the L encoding layers is

$$\begin{aligned} \Lambda &= \left\{ \sum_{l=1}^{L} \lambda_{j_l}^{(l)} \;\middle|\; j_l \in \{0, 1\} \right\} \\ &= \left\{ \frac{1}{2}\sum_{l=1}^{L} \epsilon_l s_l \;\middle|\; \epsilon_l \in \{-1, 1\} \right\}. \end{aligned} \tag{4.70}$$

Consequently, the set of accessible Fourier frequencies is

$$\begin{aligned} \Omega &= \left\{ \sum_{l=1}^{L} \left(\lambda_{j_l}^{(l)} - \lambda_{k_l}^{(l)}\right) \;\middle|\; j_l, k_l \in \{0, 1\} \right\} \\ &= \left\{ \sum_{l=1}^{L} m_l s_l \;\middle|\; m_l \in \{-1, 0, 1\} \right\}, \end{aligned} \tag{4.71}$$

which has the same form as the N-qubit spectrum with $N = L$ and $a_j = s_j$. Encoding in parallel across qubits and encoding serially through re-uploading therefore give access to the same set of frequencies.

For example, for two encoding layers with coefficient a, b, the accessible frequencies are:

$$\Omega = \{0, \pm a, \pm b, \pm(a+b), \pm(a-b)\}$$

Therefore, a circuit like following can learn $\omega = 1, 1000$:

$$U(x) = W(\theta_1)S(x)W(\theta_1)S(1000x)$$

4.5 Expressivity and Frequency Expansion

Section 4 established that a QNN computes a finite Fourier series,

$$f(x) = \sum_{\omega \in \Omega} c_\omega \, e^{i\omega x}, \tag{4.72}$$

in which the frequency spectrum Ω is fixed by the eigenvalues of the encoding Hamiltonians and the number of encoding gates, while the trainable blocks determine only the coefficients c_ω. A direct consequence is that the model can represent only functions whose frequency content lies within Ω: no choice of trainable parameters can produce a frequency outside this set. A general target function, however, may contain many frequency components, some of them high. For a QNN to act as a universal function approximator, two conditions must therefore hold: (i) the spectrum Ω must be rich enough to contain the frequencies required by the target, and (ii) the trainable blocks must be expressive enough to realize the required coefficients c_ω. Schuld et al. [35] showed that, when both conditions are met and the spectrum is allowed to grow, a QNN can approximate any square-integrable function on a bounded interval arbitrarily well. This section addresses condition (i).

Because Ω consists of differences of eigenvalue sums, it can be enlarged only by enlarging the set of eigenvalue sums. We examine three strategies for doing so:

1. *parallel encoding*, in which the input is encoded on more qubits (increasing the circuit width);

2. *serial encoding*, or data re-uploading, in which the input is encoded repeatedly on the same qubits (increasing the circuit depth); and

3. *input scaling*, in which the input is multiplied by constant coefficients before encoding, which relocates the frequencies rather than adding encoding gates.

Throughout this section, the encoding gates are single-qubit Pauli rotations. Every Pauli rotation $R_P(x) = e^{-i\frac{x}{2}P}$, with $P \in \{X, Y, Z\}$, has encoding Hamiltonian $H = \frac{1}{2}P$ and eigenvalues $\pm\frac{1}{2}$. For concreteness, we write all examples with $P = X$.

4.5.1 Parallel Encoding: Increasing the Number of Encoding Qubits

One qubit. Consider first a single encoding qubit with

$$S(x) = R_X(x) = e^{-ixH}, \qquad H = \frac{1}{2}X = \frac{1}{2}\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{4.73}$$

The eigenvalues of H are $\mathrm{spec}(H) = \{-\frac{1}{2}, \frac{1}{2}\}$, and their pairwise differences give the spectrum $\Omega = \{-1, 0, 1\}$. By the conjugate symmetry $c_{-\omega} = c_\omega^*$ established in Section 4, the model therefore reduces to a single sinusoid, $f(x) = c_0 + 2|c_1|\cos(x + \arg c_1)$.

Two qubits. Suppose now that x is encoded once on each of two qubits. Let X_q denote the Pauli-X operator acting on qubit q (and the identity on the other qubit), so that $X_1 = X \otimes I$ and $X_2 = I \otimes X$. The encoding gate is

$$S(x) = R_X(x) \otimes R_X(x) = e^{-i\frac{x}{2}X_1}\, e^{-i\frac{x}{2}X_2} = e^{-ixH}, \qquad H = \frac{1}{2}\left(X_1 + X_2\right). \tag{4.74}$$

The first equality uses $e^A \otimes e^B = (e^A \otimes I)(I \otimes e^B)$ together with $e^A \otimes I = e^{A \otimes I}$; the second holds because X_1 and X_2 commute, so that $e^A e^B = e^{A+B}$. Since X_1 and X_2 commute, they are simultaneously diagonalized by the product basis $\{|+\rangle, |-\rangle\}^{\otimes 2}$, on which X_q takes the value $\epsilon_q \in \{-1, +1\}$. The eigenvalues of H are therefore $\frac{1}{2}(\epsilon_1 + \epsilon_2)$, namely -1, 0 (twice), and 1. The set of distinct eigenvalues is $\{-1, 0, 1\}$, and its pairwise differences give the enlarged spectrum

$$\Omega = \{-2, -1, 0, 1, 2\}. \tag{4.75}$$

N qubits. The same argument applies when x is encoded once on each of N qubits. The encoding Hamiltonian is

$$H = \frac{1}{2}\sum_{q=1}^{N} X_q, \tag{4.76}$$

with eigenvalues $\frac{1}{2}\sum_q \epsilon_q$, $\epsilon_q \in \{-1, +1\}$. If k of the N signs are negative, the eigenvalue equals $\frac{1}{2}\Big((N-k) - k\Big)$, so the distinct eigenvalues are

$$\lambda_k = \frac{N}{2} - k, \qquad k = 0, 1, \ldots, N, \tag{4.77}$$

where λ_k has multiplicity $\binom{N}{k}$. Consecutive eigenvalues differ by one, and the largest and smallest differ by N; hence

$$\Omega = \{-N, -N+1, \ldots, -1, 0, 1, \ldots, N-1, N\}. \tag{4.78}$$

Encoding on N qubits thus gives access to $2N + 1$ integer frequencies. Because the spectrum is symmetric and $c_{-\omega} = c_\omega^*$, only the $N + 1$ non-negative frequencies $\{0, 1, \ldots, N\}$ are independent; the model is a real trigonometric polynomial of degree N. The number of accessible frequencies grows linearly with the number of encoding qubits.

Example. Figure 13 shows a five-qubit QNN. The unscaled input x is encoded once on each qubit through a Pauli rotation, and the circuit begins directly with this encoding layer ($W^{(1)} = I$, which does not affect the spectrum). The encoding layer is followed by two trainable layers. Each trainable layer applies parameterized single-qubit rotations R_X, R_Y, and R_Z to every qubit, followed by a ring of CNOT gates that couples each qubit to its neighbour and the last qubit back to the first, thereby generating entanglement across the register. By Eq. (4.78) with $N = 5$, the accessible frequencies are $\{0, \pm1, \pm2, \pm3, \pm4, \pm5\}$. Which of these frequencies carry non-zero coefficients in practice depends on the trainable layers.

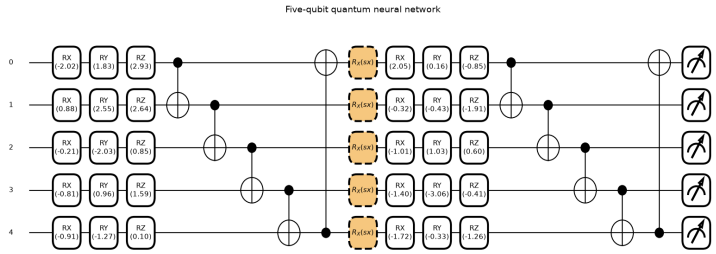


Figure 13: Architecture of the five-qubit QNN, consisting of a single encoding layer followed by two layers of parameterized rotation gates, each followed by a CNOT ring.

4.5.2 Serial Encoding: Data Re-uploading

Parallel encoding requires one qubit per encoding gate. When the number of available qubits is limited, a model with only a single qubit can still access multiple frequencies by *data re-uploading*: the same input x is encoded repeatedly on the same qubit, with trainable blocks interleaved between the encodings. For a single qubit with L encoding layers, the circuit takes the form of Eq. (4.9),

$$U(x) = W^{(L+1)} S^{(L)}(x)\, W^{(L)} \cdots W^{(2)} S^{(1)}(x)\, W^{(1)}, \tag{4.79}$$

with model output $f(x) = \langle 0|\, U^\dagger(x)\, M\, U(x)\, |0\rangle$. Here $S^{(l)}(x) = e^{-ixH_l}$ denotes the l-th encoding gate; in this subsection all encoding gates are identical, $S^{(l)}(x) = R_X(x)$ with $H_l = \frac{1}{2}X$. We allow the encoding Hamiltonians to depend on l in anticipation of Section 4.5.3.

As noted in Section 4, the derivation of the Fourier representation carries over to layer-dependent encoding Hamiltonians. Each eigenvalue sum then takes one eigenvalue from each layer, and we denote the set of all such sums by

$$\mathcal{E} = \left\{ \sum_{l=1}^{L} \lambda^{(l)} \;\middle|\; \lambda^{(l)} \in \mathrm{spec}(H_l) \right\}. \tag{4.80}$$

The spectrum consists of all pairwise differences of elements of $\mathcal{E}$:

$$\Omega = \{\Lambda - \Lambda' \mid \Lambda, \Lambda' \in \mathcal{E}\}. \tag{4.81}$$

Two encoding layers. For $L = 2$ with $H_1 = H_2 = \frac{1}{2}X$, each layer contributes an eigenvalue $\pm\frac{1}{2}$, so

$$\mathcal{E} = \left\{\pm\tfrac{1}{2} \pm \tfrac{1}{2}\right\} = \{-1, 0, 1\}, \qquad \Omega = \{-2, -1, 0, 1, 2\}. \tag{4.82}$$

This is exactly the spectrum obtained with two qubits in Section 4.5.1.

L encoding layers. For general L with $H_l = \frac{1}{2}X$ for all l, each eigenvalue sum consists of L terms equal to $\pm\frac{1}{2}$. If k of these terms are negative, the sum equals $\frac{L}{2} - k$, so that

$$\mathcal{E} = \left\{-\frac{L}{2}, -\frac{L}{2} + 1, \ldots, \frac{L}{2}\right\}, \tag{4.83}$$

and by Eq. (4.81),

$$\Omega = \{0, \pm 1, \pm 2, \ldots, \pm L\}. \tag{4.84}$$

Comparison with parallel encoding. A single qubit re-uploaded L times and $N = L$ qubits each encoded once have identical spectra. The reason is that, in both cases, every eigenvalue sum is a sum of $\pm\frac{1}{2}$ terms, one contributed by each encoding gate; the spectrum depends only on the total number of encoding gates and not on whether they are arranged in parallel or in series. The two strategies differ in their resource cost: parallel encoding consumes qubits, whereas serial encoding consumes circuit depth, which increases execution time and exposure to noise on hardware. The strategies can also be combined: encoding on N qubits in each of L layers yields NL encoding gates and hence the spectrum $\{-NL, \ldots, NL\}$.

4.5.3 Accessing High Frequencies through Input Scaling

In the encodings considered so far, the spectrum consists of consecutive integers up to the total number of encoding gates. Many applications, however, require high-frequency components to be captured. A target function containing the frequency $\omega = 1000$ would then require 1000 encoding qubits, or 1000 re-uploading layers. This cost can be avoided by multiplying the input x by a constant coefficient before it is encoded. Scaling the input rescales the encoding Hamiltonian, since $R_X(ax) = e^{-ix\frac{a}{2}X}$ corresponds to $H = \frac{a}{2}X$ with eigenvalues $\pm\frac{a}{2}$. The scaling coefficients may be fixed in advance or treated as trainable parameters; in the latter case, the spectrum itself becomes trainable.

Two qubits. Consider a two-qubit QNN in which the input to the first qubit is scaled by $a \in \mathbb{R}$ and the input to the second qubit by $b \in \mathbb{R}$. The encoding gate is

$$S(x) = R_X(ax) \otimes R_X(bx) = e^{-ixH}, \qquad H = \frac{1}{2}\left(aX_1 + bX_2\right), \tag{4.85}$$

where the second equality follows exactly as in Section 4.5.1. In the product basis $\{|+\rangle, |-\rangle\}^{\otimes 2}$, the eigenvalues of H are $\frac{1}{2}(\epsilon_1 a + \epsilon_2 b)$ with $\epsilon_1, \epsilon_2 \in \{-1, +1\}$:

$$\operatorname{spec}(H) = \left\{\frac{a+b}{2}, \ \frac{a-b}{2}, \ \frac{-a+b}{2}, \ \frac{-a-b}{2}\right\}. \tag{4.86}$$

The difference between two eigenvalues, labelled by signs (ϵ_1, ϵ_2) and $(\epsilon_1', \epsilon_2')$, is

$$\frac{\epsilon_1 - \epsilon_1'}{2}\, a + \frac{\epsilon_2 - \epsilon_2'}{2}\, b, \tag{4.87}$$

where each factor $\frac{1}{2}(\epsilon_q - \epsilon_q')$ takes a value in $\{-1, 0, 1\}$. The accessible frequencies are therefore

$$\Omega = \{0,\ \pm a,\ \pm b,\ \pm(a+b),\ \pm(a-b)\}\,. \tag{4.88}$$

High frequencies thus become accessible without additional qubits. For instance, $\omega = 1000$ lies in Ω if $a = 1000$, or if a and b are chosen such that $a + b = 1000$ or $a - b = 1000$. Setting $a = b = 1$ recovers the unscaled spectrum $\{-2, \ldots, 2\}$.

N qubits. The argument extends directly to N qubits, where qubit q encodes $a_q x$. The encoding Hamiltonian, its eigenvalues, and the resulting spectrum are

$$\begin{aligned} H &= \frac{1}{2}\sum_{q=1}^{N} a_q X_q, \\ \lambda &= \frac{1}{2}\sum_{q=1}^{N} \epsilon_q a_q, \qquad \epsilon_q \in \{-1, +1\}, \\ \Omega &= \left\{ \sum_{q=1}^{N} \nu_q a_q \;\middle|\; \nu_q \in \{-1, 0, 1\} \right\}, \end{aligned} \tag{4.89}$$

where $\nu_q = \frac{1}{2}(\epsilon_q - \epsilon_q')$ as in the two-qubit case. The unscaled encoding corresponds to $a_q = 1$ for all q, which recovers Eq. (4.78). The scaled encoding used in our experiments corresponds to $N = 5$ and $a_q = q$; since every integer from 0 to 15 is a sum of distinct elements of $\{1, \ldots, 5\}$, this choice yields $\Omega = \{-15, \ldots, 15\}$.

The frequency budget. Input scaling changes *where* the frequencies lie but not *how many* there can be. Since each ν_q in Eq. (4.89) takes one of three values, $|\Omega| \leq 3^N$ for any choice of coefficients. The choice of $\{a_q\}$ therefore trades density against reach. With $a_q = 1$, the spectrum is the dense range $\{-N, \ldots, N\}$, and many sign combinations produce the same frequency. With $a_q = q$, it extends to $\pm N(N+1)/2$. With $a_q = 3^{q-1}$, the bound is attained: every integer in $[-\frac{3^N-1}{2}, \frac{3^N-1}{2}]$ has a unique balanced-ternary representation, so Ω contains all of them, for example $\{-121, \ldots, 121\}$ when $N = 5$. Conversely, coefficients chosen to reach a single high frequency leave gaps: frequencies between those in Ω cannot be represented. If the coefficients are not commensurate, the frequencies are no longer integer multiples of a common fundamental, and f is not periodic.

Single qubit with scaled re-uploading. High frequencies can also be accessed with a single qubit by assigning each re-uploading layer in Eq. (4.79) its own coefficient. The l-th encoding gate is then

$$S^{(l)}(x) = R_X(s_l x) = e^{-ixH_l}, \qquad H_l = \frac{s_l}{2} X, \qquad \mathrm{spec}(H_l) = \left\{-\frac{s_l}{2}, \frac{s_l}{2}\right\}, \tag{4.90}$$

where $s_l \in \mathbb{R}$ is the scaling coefficient of the l-th layer. The set of eigenvalue sums is

$$\mathcal{E} = \left\{ \frac{1}{2} \sum_{l=1}^{L} \epsilon_l s_l \;\middle|\; \epsilon_l \in \{-1, +1\} \right\}, \tag{4.91}$$

and, by Eq. (4.81) and the same argument as above,

$$\Omega = \left\{ \sum_{l=1}^{L} \nu_l s_l \;\middle|\; \nu_l \in \{-1, 0, 1\} \right\}. \tag{4.92}$$

This expression has the same form as Eq. (4.89) with $N = L$ and $a_q = s_q$. As in the unscaled case, parallel encoding across qubits and serial encoding through re-uploading give access to the same set of frequencies, and the frequency budget $|\Omega| \leq 3^L$ applies.

Example. With two re-uploading layers and coefficients $s_1 = 1$ and $s_2 = 1000$, the circuit

$$U(x) = W^{(3)}\, R_X(1000x)\, W^{(2)}\, R_X(x)\, W^{(1)} \tag{4.93}$$

has the spectrum

$$\Omega = \{0, \pm 1, \pm 999, \pm 1000, \pm 1001\}. \tag{4.94}$$

A single qubit can therefore represent both the frequency $\omega = 1$ and the frequency $\omega = 1000$. The combinations $\omega = 999$ and $\omega = 1001$ are accessible as well, whereas the intermediate frequencies $2, \ldots, 998$ are not, in accordance with the frequency budget discussed above.

Summary. Table 1 summarizes the three strategies. In every case, the spectrum is determined by the set of eigenvalue sums of the encoding Hamiltonians, and the trainable blocks act only on the coefficients.

Table 1: Frequency spectra of the encoding strategies discussed in this section, for Pauli-rotation encodings. In the scaled cases, $\nu_q, \nu_l \in \{-1, 0, 1\}$.

Strategy	Encoding	Spectrum Ω	Resource cost
Parallel	N qubits, once each	$\{-N, \ldots, N\}$	N qubits
Serial (re-uploading)	1 qubit, L layers	$\{-L, \ldots, L\}$	depth L
Scaled parallel	N qubits, inputs $a_q x$	$\{\sum_q \nu_q a_q\}$	N qubits
Scaled serial	1 qubit, inputs $s_l x$	$\{\sum_l \nu_l s_l\}$	depth L

5 Applications of Quantum Neural Networks

The preceding sections established a precise characterization of what a QNN computes: a finite Fourier series whose frequencies are fixed by the data encoding and whose coefficients are learned by the trainable blocks. This section uses that characterization to address a practical question: *which tasks and applications are QNNs well suited to?* Rather than cataloguing every domain in which QNNs have been tried, we derive suitability criteria from the Fourier picture and examine representative application areas against them, distinguishing throughout what QNNs can represent in principle from what has been demonstrated in practice.

5.1 From Fourier Properties to Task Suitability

Multivariate inputs. Section 4 considered a scalar input. Most applications, however, involve an input vector $\mathbf{x} = (x_1, \ldots, x_D) \in \mathbb{R}^D$. When each feature x_k is encoded through its own Hamiltonian-generated gates, the same derivation yields a multivariate Fourier series [35, 44],

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{\boldsymbol{\omega} \in \Omega} c_{\boldsymbol{\omega}}(\boldsymbol{\theta})\, e^{i\boldsymbol{\omega}\cdot\mathbf{x}}, \qquad \Omega = \Omega_1 \times \Omega_2 \times \cdots \times \Omega_D, \tag{5.1}$$

where Ω_k is the spectrum generated by the encodings of feature x_k, and $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_D)$ is a frequency vector. If every feature is encoded so that $\Omega_k = \{-K, \ldots, K\}$, the model contains $(2K+1)^D$ frequency vectors. Both the spectrum and the number of coefficients to be controlled therefore grow exponentially with the input dimension, and Casas and Cervera-Lierta showed that, for several common circuit families, the degrees of freedom required to fit a general multidimensional Fourier series grow faster than those available in the circuit [44].

Suitability criteria. Combining Eq. (5.1) with the results of Sections 4 and 4.5, we identify five properties that make a task a natural fit for a QNN:

1. **Low input dimension.** Small D keeps the spectrum and the encoding cost manageable.
2. **Known, sparse, or structured spectral content.** When the dominant frequencies of the target are known in advance, the encoding can place Ω exactly where the signal lives, using the scaling strategies of Section 4.5.3.
3. **Smoothness or periodicity.** A truncated Fourier series represents smooth and periodic functions efficiently, whereas discontinuities produce persistent oscillations near the jump (the Gibbs phenomenon [43]).
4. **Modest data volume.** Every data point requires many circuit executions to estimate an expectation value.
5. **Quantum-native data.** When the input is itself a quantum state, the encoding problem disappears. This criterion lies outside the Fourier picture, but it is where the case for quantum advantage is strongest.

The following subsections examine application domains in roughly decreasing order of fit.

5.2 Implicit Neural Representations: Images, Audio, and Beyond

An implicit neural representation (INR) replaces a discrete signal, such as a grid of pixels or a sequence of audio samples, with a continuous function learned by a neural network. The network maps coordinates to signal values: $(x, y) \mapsto$ RGB for an image, $t \mapsto$ amplitude for an audio clip, $(x, y, z) \mapsto$ signed distance for a 3D shape [45], or position and viewing direction to colour and density in neural radiance fields [46].

INRs satisfy the first three criteria almost ideally: the input dimension is small ($D = 1$ to 5), and the quality of an INR is judged precisely by how well it reproduces the signal's

spectral content. This is also where classical networks struggle most visibly, since spectral bias causes ReLU multilayer perceptrons to produce blurry reconstructions that miss fine detail [36]. The leading classical remedies are themselves Fourier constructions: Fourier feature mappings pass the coordinates through sinusoids before an MLP [37], and SIREN uses sinusoidal activations throughout [38]. A QNN provides this structure natively, with a spectrum designed through the encoding.

This connection has been explored directly. Zhao et al. proposed the Quantum Implicit Representation Network (QIREN), which combines data re-uploading circuits with classical layers, and reported that it outperforms classical INR baselines on image and sound representation, particularly in modelling high-frequency components, while using fewer parameters; they also explored image super-resolution and image generation [47]. Because these results were obtained with classical simulation of small circuits, they establish representational promise rather than a hardware advantage.

Two obstacles remain. First, INRs are trained on every coordinate of the signal: a 256×256 image already contains 65,536 training points, each requiring many circuit executions on hardware. Second, sharp edges give rise to Gibbs oscillations in any truncated Fourier representation. Moreover, with integer spectra the model is 2π-periodic, so coordinates should be mapped to a sub-interval of $[0, 2\pi)$, as recommended for angle encoding in Section **??**, to avoid forcing the two ends of the signal to agree. Nevertheless, among classical-data applications, INRs are arguably the most natural match between task requirements and the QNN's inductive bias.

5.3 Audio and Time-Series Signals

Audio and many time series are superpositions of oscillations. Voiced and musical sounds consist of a fundamental frequency and its harmonics, and many time series exhibit known periodicities, such as daily and weekly cycles in electricity load, traffic, and transaction volumes. Classical forecasting practice already models such seasonality with explicit Fourier terms [48]. For a QNN, known periodicities translate directly into encoding design: choosing the scaling coefficients of Section 4.5.3 so that Ω contains the seasonal frequencies and their low harmonics.

Several studies have applied QNNs to such signals. Emmanoulopoulos and Dimoska reported that parameterized quantum circuits with only a few parameters forecast periodic signals with small noise components about as well as classical bidirectional LSTM networks [49]. In speech recognition, Yang et al. used quantum circuits as convolutional feature extractors on Mel-spectrograms within a hybrid model and reported competitive accuracy on a limited-vocabulary keyword-recognition benchmark [50]. In both cases the quantum component is small and evaluated in simulation.

The Fourier perspective also suggests a property that has received less attention: a fixed spectrum acts as a built-in band limit. A model whose spectrum is $\{-K, \dots, K\}$ cannot represent signal components above frequency K, so it cannot fit high-frequency noise outside its band, regardless of how the parameters are trained. This gives a concrete answer to the question raised in the Introduction: a predetermined spectrum is an advantage when the designer knows where the signal lives. Noise within the band can still be fitted, and the interplay between spectrum design, overfitting, and generalization in quantum Fourier models

is subtle [51].

5.4 Scientific Computing and Differential Equations

Spectral methods, which represent the solution of a differential equation as a truncated Fourier or Chebyshev series, are among the most accurate numerical techniques for problems with smooth solutions [52]. A QNN whose output is a truncated Fourier series can therefore be viewed as a trainable spectral solver. Physics-informed neural networks (PINNs) train a network to satisfy a differential equation by including the equation's residual in the loss [53], and the same strategy applies to QNNs. Because the QNN output is an analytic function of x, its input derivatives can be computed exactly, without finite differences.

Kyriienko, Paine, and Elfving developed this idea as *differentiable quantum circuits* (DQCs): functions are represented as expectation values of parameterized circuits, their derivatives are obtained analytically, and the circuits are trained to satisfy the equation and its boundary conditions [54]. They introduced a Chebyshev feature map and applied the method to nonlinear problems, including an instance of the Navier–Stokes equations for flow in a convergent–divergent nozzle. The Chebyshev map illustrates how the Fourier picture can be adapted to non-periodic domains: encoding $\varphi(x) = \arccos x$ instead of x turns $\cos(k\varphi)$ into the Chebyshev polynomial $T_k(x)$, so the Fourier series in φ becomes a Chebyshev series in x. Jaderberg et al. subsequently made the encoding generators trainable, allowing the model to adapt its own frequencies, and reported improved accuracy on the Navier–Stokes problem with only one additional parameter per encoding gate [55].

Scientific computing fits the criteria well when the domain is low-dimensional and the solution is smooth or oscillatory, as in wave propagation and problems with periodic boundary conditions; problems in three spatial dimensions plus time, however, quickly encounter the exponential growth of the multivariate spectrum.

5.5 High-Dimensional Classical Data: Vision, Tabular Data, and Finance

The tasks that dominate classical deep learning, such as image classification and large-scale tabular prediction, satisfy the criteria least well. A 28×28 greyscale image is a 784-dimensional input: encoding every pixel requires hundreds of qubits or aggressive dimensionality reduction, and the resulting spectrum is astronomically large while the target, a class label, bears no obvious relation to its frequency structure.

Computer vision. Practical quantum vision models are therefore hybrid: quanvolutional networks apply small quantum circuits to local image patches, like convolutional filters [56], and transfer-learning approaches place a small QNN on top of a pretrained classical feature extractor [57]. A large-scale benchmark by Bowles, Ahmed, and Schuld, however, found that out-of-the-box classical models systematically outperformed prototypical quantum classifiers on small classification tasks, that hybrid models performed similarly to their classical hosts, and that removing entanglement often did not reduce performance [58]. Claims of advantage on standard vision benchmarks should therefore be treated with caution.

Tabular data and finance. Finance has been a prominent target for quantum machine learning [59]. Fraud detection illustrates both the opportunities and the constraints. Grossi et al. applied a quantum support vector machine, a kernel-based relative of the QNN, to real card-payment data and found that results depended strongly on feature selection; the quantum model contributed complementary information within a hybrid ensemble, but only on a drastically reduced dataset that fitted the hardware of the time [60]. Fraud datasets are also large and highly imbalanced, which conflicts with the fourth criterion. The Fourier perspective nevertheless identifies a natural niche: *cyclic features*. Transaction timestamps carry hour-of-day and day-of-week patterns, and classical pipelines typically encode such features through sine and cosine transforms. A QNN represents periodic dependence on these features natively; for example, encoding $x = 2\pi h/24$ for hour h makes the model exactly periodic over a day. Relational structure in transaction networks can be addressed by quantum graph neural networks [61], although these face the same scaling constraints.

In summary, for high-dimensional classical data the realistic role of a QNN today is that of a small, specialized component within a hybrid model, operating on a compact set of features whose structure matches its inductive bias.

5.6 Quantum-Native Data

When the data are themselves quantum states, for instance the output of a quantum simulation, sensor, or physical experiment, no classical-to-quantum encoding is required and the obstacles associated with criteria 1 and 4 largely disappear. Cong, Choi, and Lukin introduced quantum convolutional neural networks and showed that they can recognize quantum phases of matter using a number of parameters that grows only logarithmically with system size [62]. Huang et al. proved, and demonstrated experimentally, that for certain tasks learning from quantum experiments can require exponentially fewer experiments when quantum data are processed coherently by a quantum computer than when they are first measured and processed classically [63]. In addition, quantum models have been shown to generalize from relatively few training examples [64]. For quantum data, the question of advantage rests on firmer theoretical ground than for any classical-data application discussed above.

5.7 Limitations and Outlook

Data loading and measurement cost. Many proposed quantum speedups assume that classical data can be loaded into quantum states efficiently, an assumption that often dominates the true cost [65]. At the output, estimating an expectation value to precision ϵ requires on the order of $1/\epsilon^2$ circuit executions, so every prediction carries a sampling overhead that classical networks do not incur.

Trainability and noise. Barren plateaus make gradients vanish exponentially in the number of qubits for sufficiently random circuits [31], and hardware noise can induce barren plateaus of its own [66].

Table 2: Suitability of QNNs across application domains, assessed against the criteria of Section 5.1.

Domain	Input dim.	Spectral structure	Fit	Main obstacle
Implicit neural representations	1–5	Rich but structured; edges are problematic	High	Number of coordinates; Gibbs oscillations at edges
Audio and time series	1–few	Known periodicities and harmonics	High	Broadband noise and non-stationarity
Differential equations	1–4	Smooth or oscillatory solutions	High	Multivariate spectrum growth in higher dimensions
Computer vision	10^2–10^6	Weakly related to the target	Low	Encoding cost; classical baselines are strong
Tabular data and finance	10–10^3	Mostly unstructured, except cyclic features	Medium	Large, imbalanced datasets
Quantum-native data	n/a	Outside the Fourier framework	Highest	Access to quantum data sources

Constrained expressivity. A large spectrum does not imply that all coefficients are freely adjustable. Mhiri et al. showed that the variance of a Fourier coefficient depends on the redundancy of its frequency, that is, on how many pairs of eigenvalue sums produce it, and that some coefficients can vanish exponentially as the number of qubits grows [67]. For uniform Pauli encodings, redundancy is highest at low frequencies, which suggests that QNNs can exhibit a form of spectral bias of their own. Exponential encodings [68, 69] reduce the number of gates needed for a given spectrum but also reduce redundancy, a trade-off that should be assessed explicitly.

Classical surrogates and dequantization. The Fourier structure that makes QNNs interpretable also makes them vulnerable to classical approximation. If the spectrum is known, a classical model can fit the same function class, either by constructing a classical surrogate from the quantum model [70] or by sampling frequencies with random Fourier features [33]. Sweke et al. characterized the conditions under which random Fourier features efficiently dequantize variational QML for regression [71]. Consequently, the applications most worth pursuing are those in which the relevant spectrum is hard to sample classically or the data are quantum-native.

Outlook. Table 2 summarizes the analysis. The domains with the best fit share low-dimensional inputs, structured spectral content, and moderate data volume: INRs, periodic signal modelling, and spectral solvers for differential equations are the most promising classical-data applications, and quantum-native data the most promising overall. For high-dimensional classical data, QNNs are best viewed as specialized components within hybrid architectures. The Fourier perspective thus offers more than a description of what QNNs compute; it provides a principled way to decide where they should be applied.

References

[1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.

[2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, pages 1097–1105, 2012.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[6] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.

[7] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

[8] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[10] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[11] Jaime Sevilla, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. Compute trends across three eras of machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2022.

[12] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019.

[13] International Energy Agency. Key questions on energy and AI. Technical report, IEA, Paris, 2026.

[14] Richard P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6/7):467–488, 1982.

[15] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[16] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219, 1996.

[17] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[18] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

[19] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C. Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando G. S. L. Brandao, David A. Buell, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, 2019.

[20] Google Quantum AI and Collaborators. Quantum error correction below the surface code threshold. *Nature*, 638(8052):920–926, 2025.

[21] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.

[22] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549:195–202, 2017.

[23] Maria Schuld and Francesco Petruccione. *Machine Learning with Quantum Computers*. Springer, 2nd edition, 2021.

[24] Edward Farhi and Hartmut Neven. Classification with quantum neural networks on near term processors. *arXiv preprint arXiv:1802.06002*, 2018.

[25] Kosuke Mitarai, Makoto Negoro, Masahiro Kitagawa, and Keisuke Fujii. Quantum circuit learning. *Physical Review A*, 98(3):032309, 2018.

[26] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, 2019.

[27] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212, 2019.

[28] Maria Schuld and Nathan Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122(4):040504, 2019.

[29] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, 2020.

[30] Amira Abbas, David Sutter, Christa Zoufal, Aurélien Lucchi, Alessio Figalli, and Stefan Woerner. The power of quantum neural networks. *Nature Computational Science*, 1:403–409, 2021.

[31] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018.

[32] Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R. McClean. Power of data in quantum machine learning. *Nature Communications*, 12:2631, 2021.

[33] Jonas Landman, Slimane Thabet, Constantin Dalyac, Hela Mhiri, and Elham Kashefi. Classically approximating variational quantum machine learning with random Fourier features. In *International Conference on Learning Representations (ICLR)*, 2023.

[34] M. Cerezo, Martin Larocca, Diego García-Martín, N. L. Diaz, Paolo Braccia, Enrico Fontana, Manuel S. Rudolph, Pablo Bermejo, Aroosa Ijaz, Supanut Thanasilp, Eric R. Anschuetz, and Zoë Holmes. Does provable absence of barren plateaus imply classical simulability? *Nature Communications*, 16:7907, 2025.

[35] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3):032430, 2021.

[36] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5301–5310. PMLR, 2019.

[37] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems*, volume 33, pages 7537–7547, 2020.

[38] Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[39] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[40] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[41] Matus Telgarsky. Benefits of depth in neural networks. In *29th Annual Conference on Learning Theory*, volume 49 of *Proceedings of Machine Learning Research*, pages 1517–1539. PMLR, 2016.

[42] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. Elementary gates for quantum computation. *Physical Review A*, 52(5):3457–3467, 1995.

[43] Elias M. Stein and Rami Shakarchi. *Fourier Analysis: An Introduction*, volume 1 of *Princeton Lectures in Analysis*. Princeton University Press, 2003.

[44] Berta Casas and Alba Cervera-Lierta. Multidimensional Fourier series with quantum circuits. *Physical Review A*, 107:062612, 2023.

[45] Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 165–174, 2019.

[46] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421, 2020.

[47] Jiaming Zhao, Wenbo Qiao, Peng Zhang, and Hui Gao. Quantum implicit neural representations. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 60940–60956. PMLR, 2024.

[48] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021.

[49] Dimitrios Emmanoulopoulos and Sofija Dimoska. Quantum machine learning in finance: Time series forecasting. *arXiv preprint arXiv:2202.00599*, 2022.

[50] Chao-Han Huck Yang, Jun Qi, Samuel Yen-Chi Chen, Pin-Yu Chen, Sabato Marco Siniscalchi, Xiaoli Ma, and Chin-Hui Lee. Decentralizing feature extraction with quantum convolutional neural network for automatic speech recognition. In *2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6523–6527, 2021.

[51] Evan Peters and Maria Schuld. Generalization despite overfitting in quantum machine learning models. *Quantum*, 7:1210, 2023.

[52] John P. Boyd. *Chebyshev and Fourier Spectral Methods*. Dover Publications, 2nd edition, 2001.

[53] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[54] Oleksandr Kyriienko, Annie E. Paine, and Vincent E. Elfving. Solving nonlinear differential equations with differentiable quantum circuits. *Physical Review A*, 103:052416, 2021.

[55] Ben Jaderberg, Antonio A. Gentile, Youssef Achari Berrada, Elvira Shishenina, and Vincent E. Elfving. Let quantum neural networks choose their own frequencies. *arXiv preprint arXiv:2309.03279*, 2023.

[56] Maxwell Henderson, Samriddhi Shakya, Shashindra Pradhan, and Tristan Cook. Quanvolutional neural networks: Powering image recognition with quantum circuits. *Quantum Machine Intelligence*, 2:2, 2020.

[57] Andrea Mari, Thomas R. Bromley, Josh Izaac, Maria Schuld, and Nathan Killoran. Transfer learning in hybrid classical-quantum neural networks. *Quantum*, 4:340, 2020.

[58] Joseph Bowles, Shahnawaz Ahmed, and Maria Schuld. Better than classical? The subtle art of benchmarking quantum machine learning models. *arXiv preprint arXiv:2403.07059*, 2024.

[59] Dylan Herman, Cody Googin, Xiaoyuan Liu, Yue Sun, Alexey Galda, Ilya Safro, Marco Pistoia, and Yuri Alexeev. Quantum computing for finance. *Nature Reviews Physics*, 5:450–465, 2023.

[60] Michele Grossi, Noelle Ibrahim, Voica Radescu, Robert Loredo, Kirsten Voigt, Constantin von Altrock, and Andreas Rudnik. Mixed quantum–classical method for fraud detection with quantum feature selection. *IEEE Transactions on Quantum Engineering*, 3:1–12, 2022.

[61] Guillaume Verdon, Trevor McCourt, Enxhell Luzhnica, Vikash Singh, Stefan Leichenauer, and Jack Hidary. Quantum graph neural networks. *arXiv preprint arXiv:1909.12264*, 2019.

[62] Iris Cong, Soonwon Choi, and Mikhail D. Lukin. Quantum convolutional neural networks. *Nature Physics*, 15:1273–1278, 2019.

[63] Hsin-Yuan Huang, Michael Broughton, Jordan Cotler, Sitan Chen, Jerry Li, Masoud Mohseni, Hartmut Neven, Ryan Babbush, Richard Kueng, John Preskill, and Jarrod R. McClean. Quantum advantage in learning from experiments. *Science*, 376(6598):1182–1186, 2022.

[64] Matthias C. Caro, Hsin-Yuan Huang, M. Cerezo, Kunal Sharma, Andrew Sornborger, Lukasz Cincio, and Patrick J. Coles. Generalization in quantum machine learning from few training data. *Nature Communications*, 13:4919, 2022.

[65] Scott Aaronson. Read the fine print. *Nature Physics*, 11:291–293, 2015.

[66] Samson Wang, Enrico Fontana, M. Cerezo, Kunal Sharma, Akira Sone, Lukasz Cincio, and Patrick J. Coles. Noise-induced barren plateaus in variational quantum algorithms. *Nature Communications*, 12:6961, 2021.

[67] Hela Mhiri, Léo Monbroussou, Mario Herrero-González, Slimane Thabet, Elham Kashefi, and Jonas Landman. Constrained and vanishing expressivity of quantum Fourier models. *Quantum*, 9:1847, 2025.

[68] S. Shin, Y. S. Teo, and H. Jeong. Exponential data encoding for quantum supervised learning. *Physical Review A*, 107:012422, 2023.

[69] Mo Kordzanganeh, Pavel Sekatski, Leonid Fedichkin, and Alexey Melnikov. An exponentially-growing family of universal quantum circuits. *Machine Learning: Science and Technology*, 4(3):035036, 2023.

[70] Franz J. Schreiber, Jens Eisert, and Johannes Jakob Meyer. Classical surrogates for quantum learning models. *Physical Review Letters*, 131:100803, 2023.

[71] Ryan Sweke, Erik Recio-Armengol, Sofiene Jerbi, Elies Gil-Fuster, Bryce Fuller, Jens Eisert, and Johannes Jakob Meyer. Potential and limitations of random Fourier features for dequantizing quantum machine learning. *Quantum*, 9:1640, 2025.